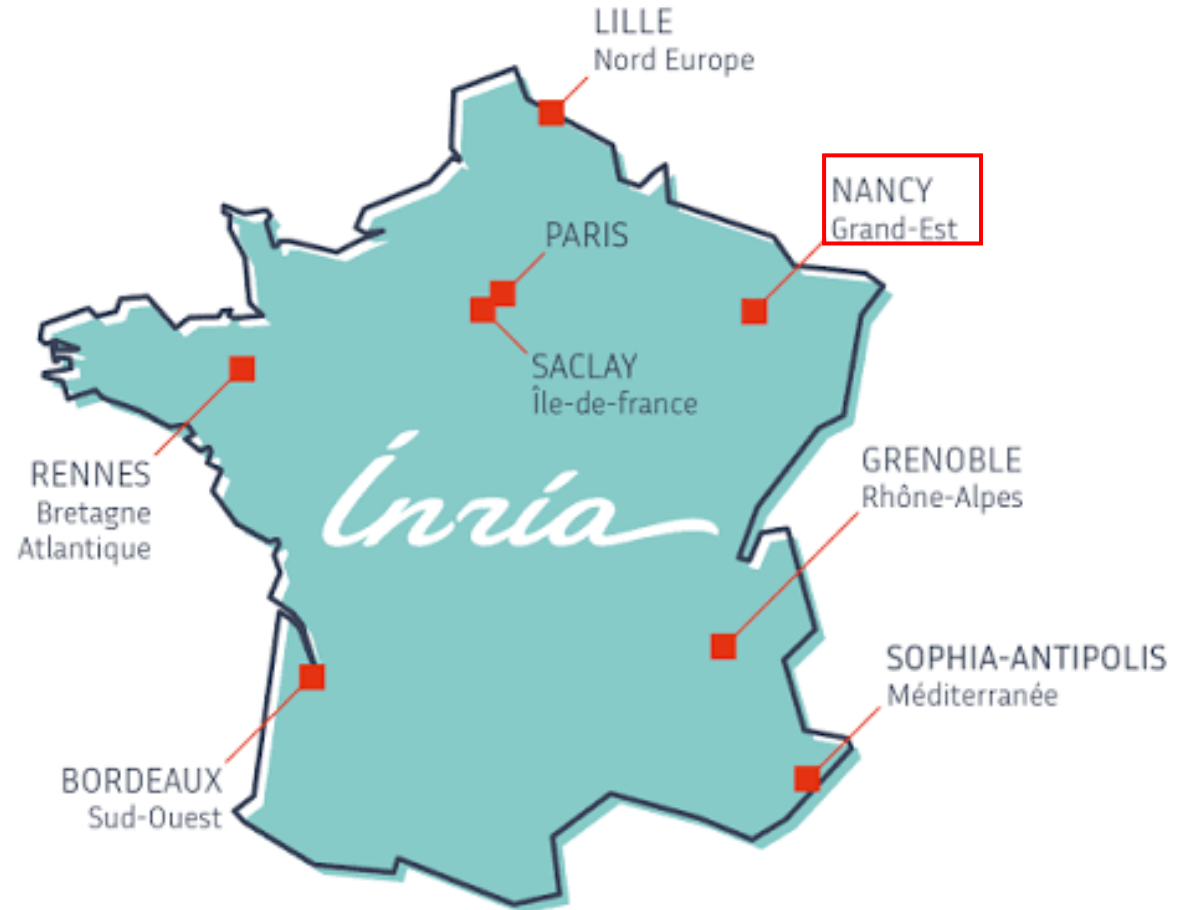


From Reactive to Predictive Control

Enrico Mingo Hoffman, Researcher Inria Université de Lorraine
enrico.mingo-hoffman@inria.fr



INRIA: National Institute for Research in Digital Science and Technology



HuCeBot Team: <https://team.inria.fr/hucebot/>

Optimization for Robotics Summer School



- Organized by the University of Patras, Greece, Summer 2025
- Lectures and code on Optimal Control



Gold Sponsors



Bronze Sponsors



Diamond Sponsors



Platinum Sponsors



Reactive Vs. Predictive Control

- Reactive Control

- **No Horizon:** only 1-step control input is computed

- Used to map **instantaneous** Operational Space commands into Joint Space commands

- Predictive Control

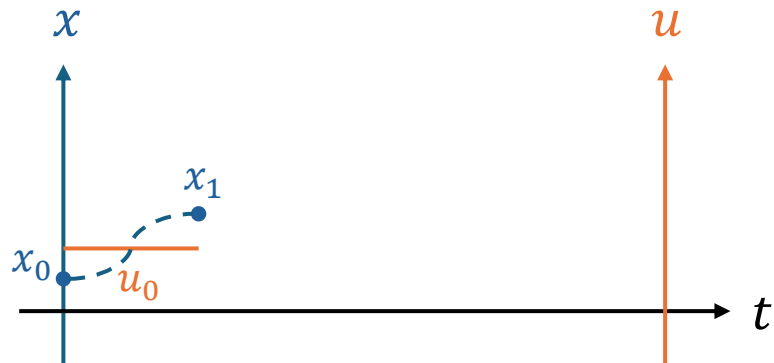
- **Horizon:** N control input and N+1 states are computed

- Used to stabilize unstable systems

- Both uses Optimization techniques

Reactive Vs. Predictive Control

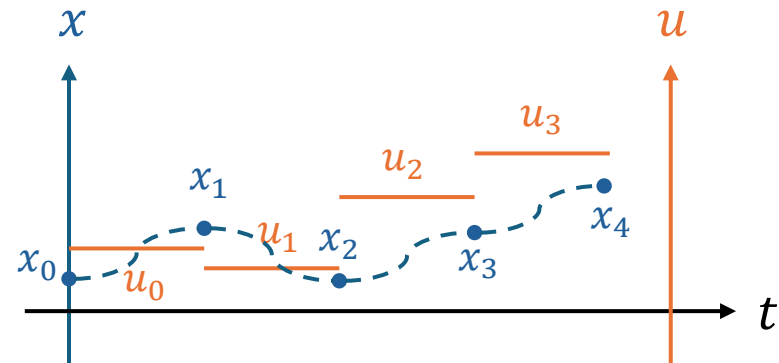
- Reactive Control



- ✓ Cheap to compute
- ✓ Easy to implement

- x Can not exploit future information
- x Requires a high-level planner

- Predictive Control

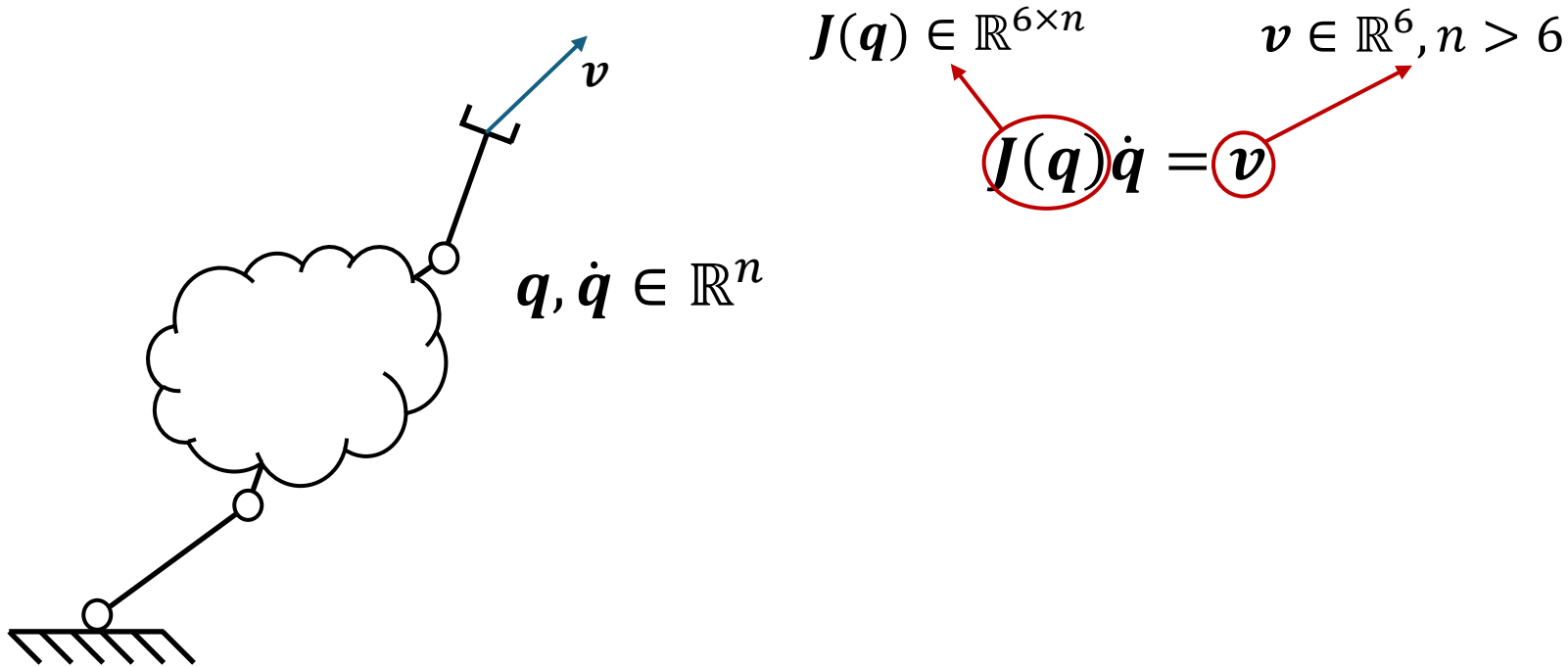


- ✓ Can exploit future information
- ✓ Stabilize

- x (More) Expensive to compute
- x Complex to implement

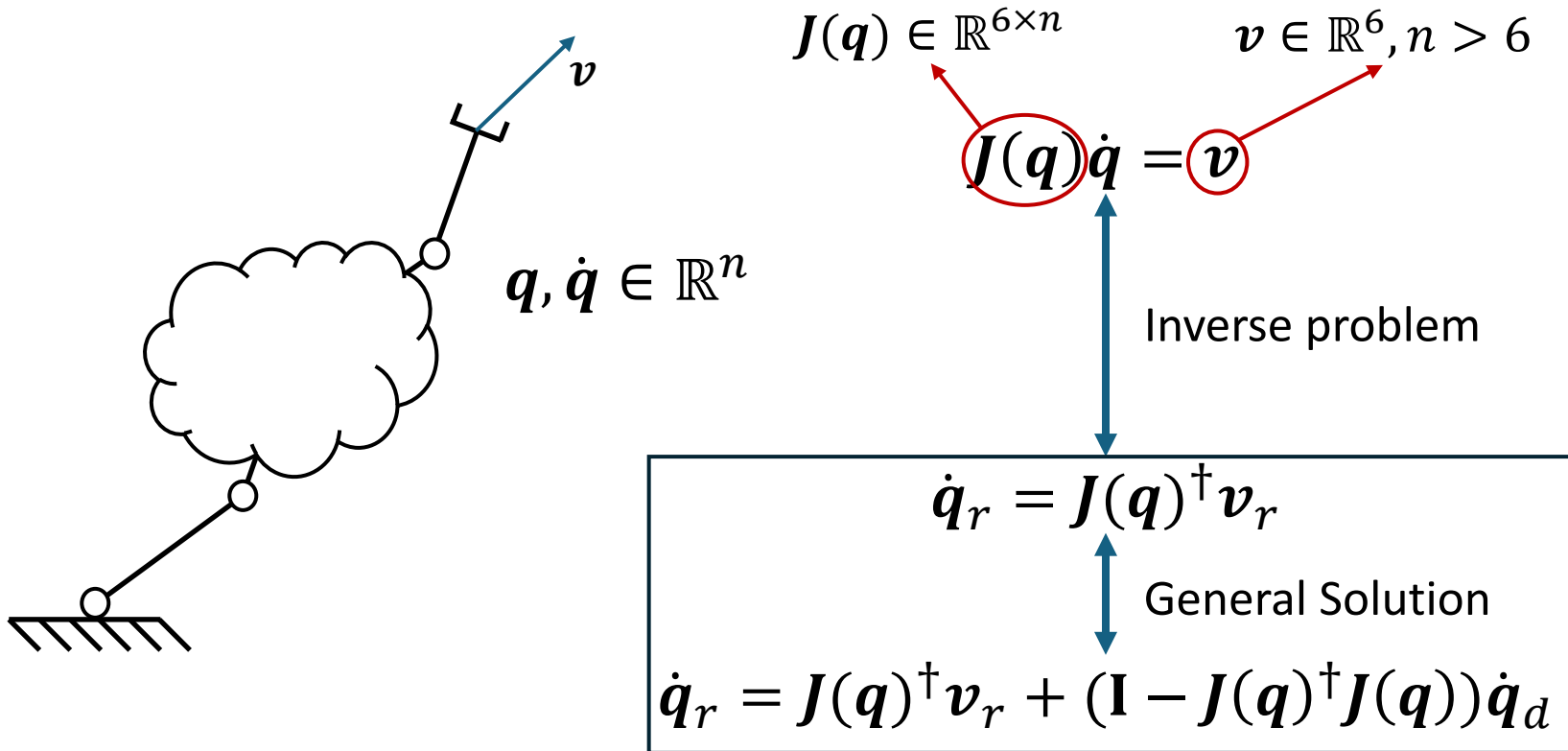
Differential Inverse Kinematics

- Instantaneous mapping from Cartesian- to Joint-space



Differential Inverse Kinematics

- Instantaneous mapping from Cartesian- to Joint-space



Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{aligned} & \mathbf{W} \in \mathbb{R}^{n \times n} \\ \min_{\dot{\mathbf{q}}} & \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_{\mathbf{W}}^2 \\ \text{s.t.} & \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} \end{aligned}$$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{array}{ll} \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_{\mathbf{W}}^2 & \xrightarrow{\quad} \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda}) = \frac{1}{2} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}})^T \mathbf{W} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + \boldsymbol{\lambda}^T (\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}) \\ \text{s.t. } \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} & \end{array}$$

$\mathbf{W} \in \mathbb{R}^{n \times n}$

$\boldsymbol{\lambda} \in \mathbb{R}^6$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{array}{ll} \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_{\mathbf{W}}^2 & \xrightarrow{\quad} \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda}) = \frac{1}{2} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}})^T \mathbf{W} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + \boldsymbol{\lambda}^T (\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}) \\ \text{s.t. } \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} & \end{array}$$

$\mathbf{W} \in \mathbb{R}^{n \times n}$ $\boldsymbol{\lambda} \in \mathbb{R}^6$

KKT System:

$$\begin{cases} \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \dot{\mathbf{q}}} = -\mathbf{W}\dot{\mathbf{q}}_d + \mathbf{W}\dot{\mathbf{q}} + \mathbf{J}^T \boldsymbol{\lambda} = \mathbf{0} & \textcircled{1} \\ \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \boldsymbol{\lambda}} = \mathbf{J}\dot{\mathbf{q}} - \mathbf{v}_d = \mathbf{0} & \textcircled{2} \end{cases}$$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{aligned} \min_{\dot{\mathbf{q}}} \quad & \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_{\mathbf{W}}^2 \quad \xrightarrow{\quad} \quad \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda}) = \frac{1}{2} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}})^T \mathbf{W} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + \boldsymbol{\lambda}^T (\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}) \\ \text{s.t.} \quad & \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} \end{aligned}$$

$\mathbf{W} \in \mathbb{R}^{n \times n}$ (pointing to $\mathbf{W}$) $\boldsymbol{\lambda} \in \mathbb{R}^6$ (pointing to $\boldsymbol{\lambda}$)

KKT System:

$$\begin{cases} \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \dot{\mathbf{q}}} = -\mathbf{W}\dot{\mathbf{q}}_d + \mathbf{W}\dot{\mathbf{q}} + \mathbf{J}^T \boldsymbol{\lambda} = \mathbf{0} & \textcircled{1} \\ \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \boldsymbol{\lambda}} = \mathbf{J}\dot{\mathbf{q}} - \mathbf{v}_d = \mathbf{0} & \textcircled{2} \end{cases}$$

from $\textcircled{1}$

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}_d - \mathbf{W}^{-1} \mathbf{J}^T \boldsymbol{\lambda} \quad \textcircled{3}$$

substituted in $\textcircled{2}$

$$\boldsymbol{\lambda} = (\mathbf{J} \mathbf{W}^{-1} \mathbf{J}^T)^{-1} (-\mathbf{v}_d + \mathbf{J}\dot{\mathbf{q}}_d)$$

substituted in $\textcircled{3}$

$$\dot{\mathbf{q}} = \mathbf{W}^{-1} \mathbf{J}^T (\mathbf{J} \mathbf{W}^{-1} \mathbf{J}^T)^{-1} \mathbf{v}_d + (\mathbf{I} - \mathbf{W}^{-1} \mathbf{J}^T (\mathbf{J} \mathbf{W}^{-1} \mathbf{J}^T)^{-1} \mathbf{J}) \dot{\mathbf{q}}_d$$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{aligned} \min_{\dot{\mathbf{q}}} \quad & \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_{\mathbf{W}}^2 \quad \xrightarrow{\quad} \quad \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda}) = \frac{1}{2} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}})^T \mathbf{W} (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + \boldsymbol{\lambda}^T (\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}) \\ \text{s.t.} \quad & \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} \end{aligned}$$

$\mathbf{W} \in \mathbb{R}^{n \times n}$ (pointing to $\mathbf{W}$) $\boldsymbol{\lambda} \in \mathbb{R}^6$ (pointing to $\boldsymbol{\lambda}$)

KKT System:

$$\begin{cases} \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \dot{\mathbf{q}}} = -\mathbf{W}\dot{\mathbf{q}}_d + \mathbf{W}\dot{\mathbf{q}} + \mathbf{J}^T \boldsymbol{\lambda} = \mathbf{0} & \textcircled{1} \\ \frac{\partial \mathcal{L}(\dot{\mathbf{q}}, \boldsymbol{\lambda})}{\partial \boldsymbol{\lambda}} = \mathbf{J}\dot{\mathbf{q}} - \mathbf{v}_d = \mathbf{0} & \textcircled{2} \end{cases}$$

from $\textcircled{1}$

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}_d - \mathbf{W}^{-1} \mathbf{J}^T \boldsymbol{\lambda} \quad \textcircled{3}$$

substituted in $\textcircled{2}$

$$\boldsymbol{\lambda} = (\mathbf{J} \mathbf{W}^{-1} \mathbf{J}^T)^{-1} (-\mathbf{v}_d + \mathbf{J}\dot{\mathbf{q}}_d)$$

substituted in $\textcircled{3}$

$$\xrightarrow{\quad \mathbf{W} = \mathbf{I} \quad} \dot{\mathbf{q}} = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T)^{-1} \mathbf{v}_d + (\mathbf{I} - \mathbf{J}^T (\mathbf{J} \mathbf{J}^T)^{-1} \mathbf{J}) \dot{\mathbf{q}}_d$$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{aligned} & \min_{\dot{q}} \frac{1}{2} \|\dot{q}_d - \dot{q}\|_W^2 \quad \xrightarrow{\quad} \quad \mathcal{L}(\dot{q}, \lambda) = \frac{1}{2} (\dot{q}_d - \dot{q})^T W (\dot{q}_d - \dot{q}) + \lambda^T (J\dot{q} - v) \\ & \text{s.t. } J(q)\dot{q} = v \end{aligned}$$

$W \in \mathbb{R}^{n \times n}$ $\lambda \in \mathbb{R}^6$

KKT System:

$$\begin{cases} \frac{\partial \mathcal{L}(\dot{q}, \lambda)}{\partial \dot{q}} = -W\dot{q}_d + W\dot{q} + J^T \lambda = \mathbf{0} & \textcircled{1} \\ \frac{\partial \mathcal{L}(\dot{q}, \lambda)}{\partial \lambda} = J\dot{q} - v_d = \mathbf{0} & \textcircled{2} \end{cases}$$



$$\begin{aligned} & \min_x \frac{1}{2} \|Ax - b\|_W^2 \\ & \text{s.t. } Cx = c \end{aligned}$$

$$\mathcal{L}(x, \lambda) = \frac{1}{2} x^T \mathbf{A}^T \mathbf{W} \mathbf{A} x - \mathbf{b}^T \mathbf{W} \mathbf{A} x + \frac{1}{2} \mathbf{b}^T \mathbf{b} + \lambda^T (Cx - c)$$

$$\Rightarrow \begin{bmatrix} \mathbf{H} & \mathbf{C}^T \\ \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -\mathbf{g} \\ c \end{bmatrix}$$

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\begin{aligned} \min_{\dot{\mathbf{q}}} & \frac{1}{2} \|\dot{\mathbf{q}}_d - \dot{\mathbf{q}}\|_W^2 \\ \text{s.t.} & \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v} \end{aligned}$$

- More flexible: we can consider linear constraints in the form of equalities and inequalities:

$$\begin{aligned} \mathbf{C}(\mathbf{q})\dot{\mathbf{q}} &= \mathbf{c} \\ \mathbf{d}_m &\leq \mathbf{D}(\mathbf{q})\dot{\mathbf{q}} \leq \mathbf{d}_M \end{aligned}$$

- Hierarchies of tasks can be implemented: *iHQP* [1], *nHQP* [2], *HCOD* [3]

[1] O. Kanoun, F. Lamiriaux, and P.-B. Wieber, "Kinematic control of redundant manipulators: Generalizing the task-priority framework to inequality task," IEEE Transactions on Robotics, vol. 27, no. 4, pp. 785–792, 2011.

[2] M. de Lasa and A. Hertzmann, "Prioritized optimization for task-space control," in IEEE/RSJ International Conference on Intelligent Robots and Systems, 2009, pp. 5755–5762.

[3] A. Escande, N. Mansard, and P.-B. Wieber, "Hierarchical quadratic programming: Fast online humanoid-robot motion generation," The International Journal of Robotics Research, vol. 33, no. 7, pp. 10061028, 2014.

Differential Inverse Kinematics

- Solution of an equivalent QP Problem:

$$\min_{\dot{\mathbf{q}}} \frac{1}{2} (\|\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}\|_{\mathbf{W}}^2 + \varepsilon \|\dot{\mathbf{q}}\|) \quad \text{Tikhonov Regularization}$$

s.t. ...

Without Regularization

$$\mathcal{F} = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{J}^T \mathbf{W} \mathbf{J} \dot{\mathbf{q}} + (\mathbf{J}\dot{\mathbf{q}})^T \mathbf{W} \mathbf{v}, \quad \mathbf{H} = \mathbf{J}^T \mathbf{W} \mathbf{J} \in \mathbb{R}^{n \times n}, \quad \text{rank}(\mathbf{H}) = 6$$

With Regularization

$$\mathcal{F} = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{J}^T \mathbf{W} \mathbf{J} \dot{\mathbf{q}} + \frac{\varepsilon}{2} \dot{\mathbf{q}}^T \dot{\mathbf{q}} + (\mathbf{J}\dot{\mathbf{q}})^T \mathbf{W} \mathbf{v}, \quad \mathbf{H} = \mathbf{J}^T \mathbf{W} \mathbf{J} + \varepsilon \mathbf{I} \in \mathbb{R}^{n \times n}, \quad \text{rank}(\mathbf{H}) = n$$

Considerations

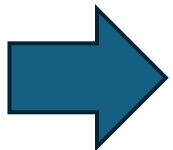
- Inverse Kinematics (CLIK [1]):

$$\mathbf{v} = \mathbf{0} + \lambda \begin{bmatrix} \mathbf{p}_d - \mathbf{p} \\ \eta \boldsymbol{\epsilon}_d - \eta_d \boldsymbol{\epsilon} - [\boldsymbol{\epsilon}_d \times] \boldsymbol{\epsilon} \end{bmatrix}$$

$$\mathbf{a} = \mathbf{0} + \lambda_1(\mathbf{0} - \mathbf{v}) + \lambda_2 \begin{bmatrix} \mathbf{p}_d - \mathbf{p} \\ \eta \boldsymbol{\epsilon}_d - \eta_d \boldsymbol{\epsilon} - [\boldsymbol{\epsilon}_d \times] \boldsymbol{\epsilon} \end{bmatrix}$$



$$\begin{array}{ll} \min_{\Delta \mathbf{q}} & \frac{1}{2} \|\mathbf{J}(\mathbf{q})\Delta \mathbf{q} - \mathbf{v}\| + \varepsilon \|\Delta \mathbf{q}\| \\ \text{s.t.} & \dots \end{array}$$



$$\mathbf{q}_{k+1} = \mathbf{q}_k + \Delta \mathbf{q}_k \longrightarrow f(\mathbf{q}^*) = \mathbf{p}_d$$

Warning: if quaternion product < 0 : flip one quaternion to ensure shortest path error.

Considerations on Constraints

- Inequalities:
 - Small to Medium problem: **Active Set** methods [1] (good for reactive control)
 - Large: **ADMM** [2], **Interior Point** [3] (good for TO and MPC)
- Pay attention to control bounds and imposed integral constraints with a relative degree > 1 !

Example: position bounds and velocity bounds in acceleration-based DIK

Some methods are [4]:

- pStep-Ahead Predictor (PSAP)
- Control Barrier Functions (CBF)
- *Viability and Invariance*

[1] Ferreau, Hans Joachim, et al. "qpOASES: A parametric active-set algorithm for quadratic programming." *Mathematical Programming Computation* 6.4 (2014)

[2] Stellato, Bartolomeo, et al. "OSQP: an operator splitting solver for quadratic programs: B. Stellato et al." *Mathematical Programming Computation* 12.4 (2020)

[3] A. Wächter and L. T. Biegler, On the Implementation of a Primal-Dual Interior Point Filter Line Search Algorithm for Large-Scale Nonlinear Programming, *Mathematical Programming* 106(1), pp. 25-57, 2006

[4] Testa, A., Raiano, L., Laghi, M., Ajoudani, A., Mingo Hoffman, E. (2024). Joint Position Bounds in Resolved-Acceleration Control: A Comparison. In: Human-Friendly Robotics 2023. HFR 2023. Springer Proceedings in Advanced Robotics, vol 29. Springer

Whole-Body Control

- Dynamics:

$$\mathbf{q} = \begin{bmatrix} \mathbf{q}_b \\ \mathbf{q}_j \end{bmatrix} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\rho}_b \\ \mathbf{q}_j \end{bmatrix}, \quad \|\boldsymbol{\rho}_b\| = 1$$

$$\mathbf{v} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \boldsymbol{\omega} \\ \dot{\mathbf{q}}_j \end{bmatrix}$$

$$\mathbf{f} = \begin{bmatrix} \mathbf{f}_0 \\ \vdots \\ \mathbf{f}_{C-1} \end{bmatrix}$$

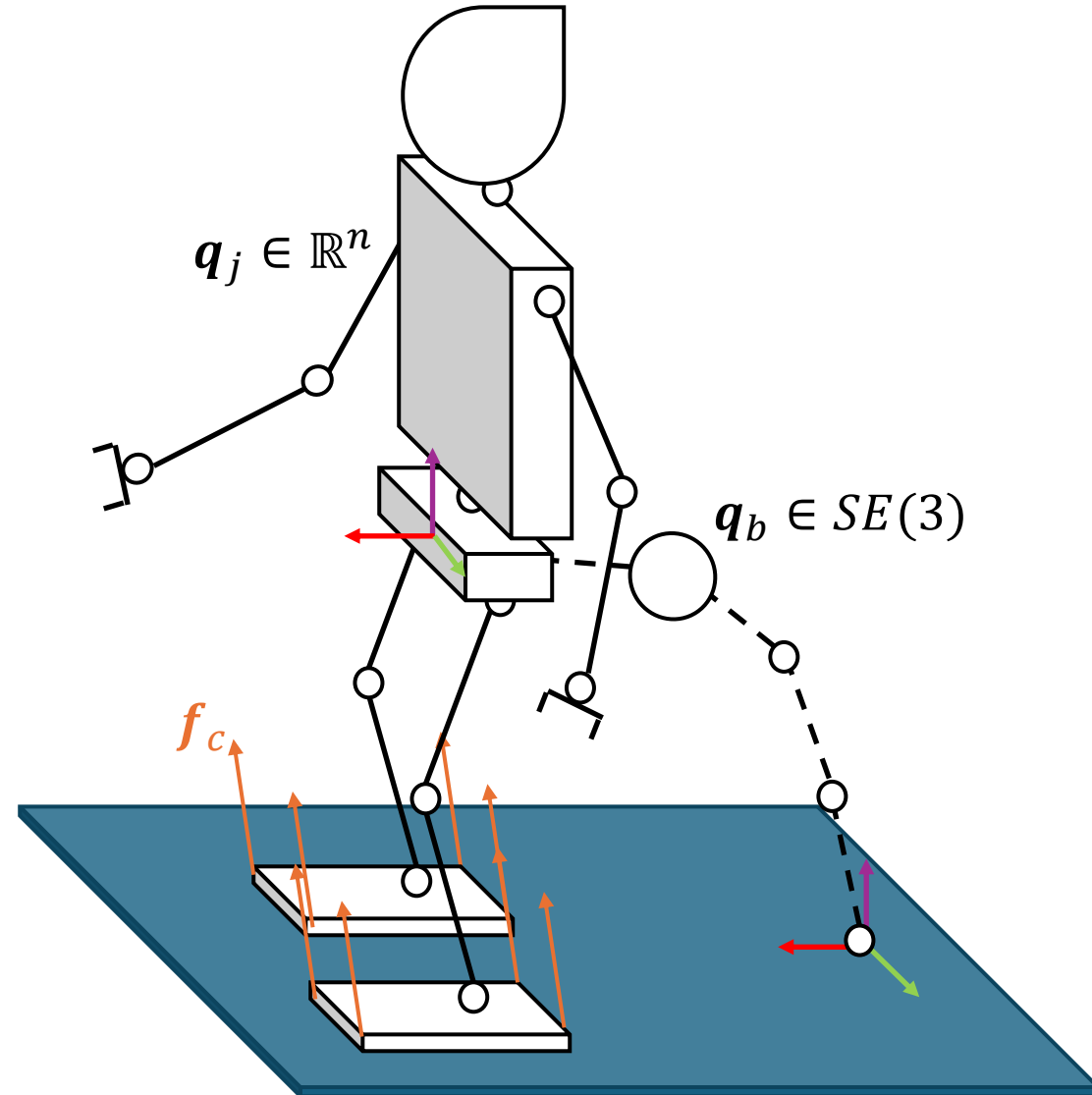
$$\min_{\dot{\mathbf{v}}, \mathbf{f}} \frac{1}{2} \left(\sum_{i=0}^{T-1} \|J_i \dot{\mathbf{v}} - \bar{\mathbf{a}}_i\|_{W_i} + \varepsilon \|\boldsymbol{\tau}\| \right)$$

$$\text{s.t. } J_c \dot{\mathbf{v}} = -\dot{J}_c \mathbf{v}, \forall c$$

$$\mathbf{M}_b \dot{\mathbf{v}} + \mathbf{h}_b = \sum_{c=0}^{C-1} J_{c,b}^T \mathbf{f}_c$$

$$\longrightarrow \mathbf{f}_c \mathbf{n} \geq 0, \forall c \longleftarrow$$

$$\boldsymbol{\tau} = \mathbf{M}_j \ddot{\mathbf{q}} + \mathbf{h}_j - \sum_{c=0}^{C-1} J_{c,j}^T \mathbf{f}_c$$

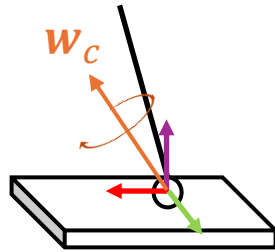


Whole-Body Control

- Consideration: Surface Contact Modeling

- *Contact Wrench*

$$w_c = \begin{bmatrix} f_c \\ \tau_c \end{bmatrix}$$

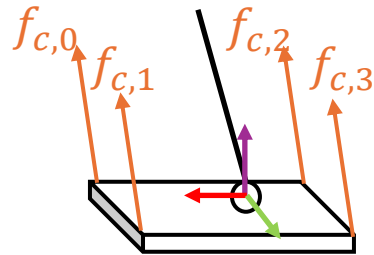


- 6 variables (3 forces, 3 torques)
- 1 Friction Cone constraint (4 rows)
- 1 Unilaterality of Normal force constraint (1 row)
- 1 Center of Pressure constraint (4 rows)
- 1 Normal torque bound (8 rows) [1]

} 17 constraints

- *Contact Forces*

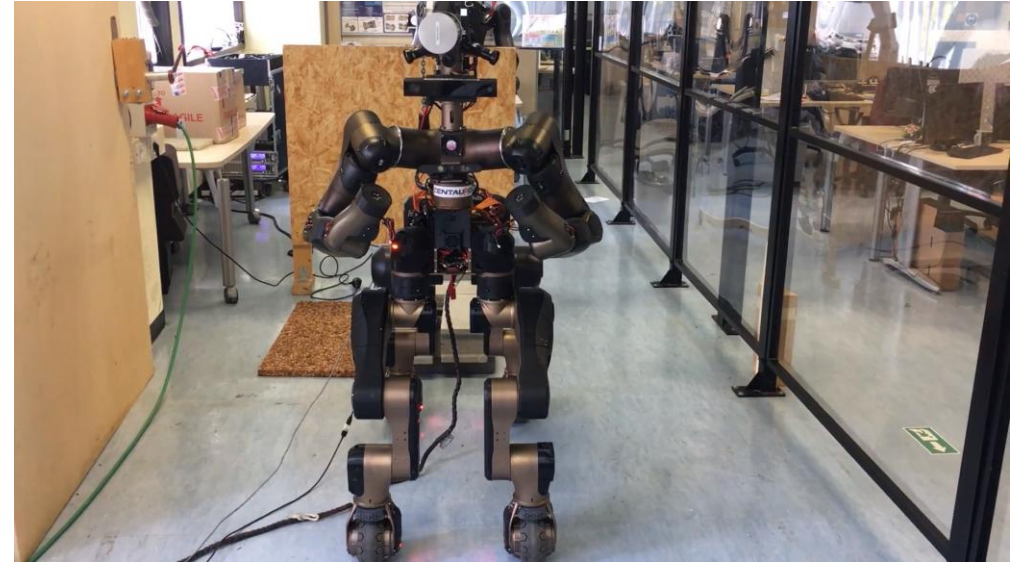
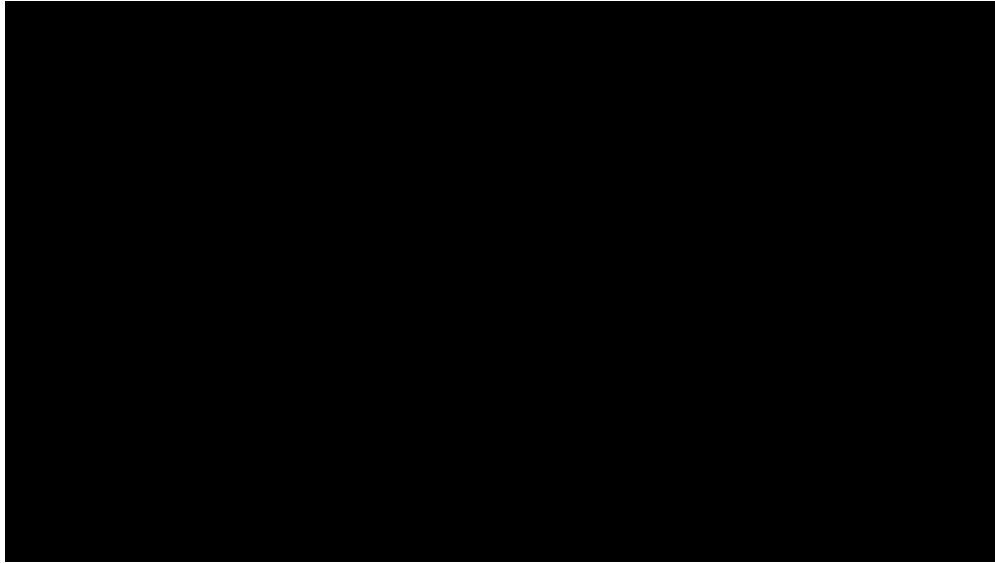
$$f_c = \begin{bmatrix} f_{c,0} \\ f_{c,1} \\ f_{c,2} \\ f_{c,3} \end{bmatrix}$$



- 12 variables (12 forces)
- 4 Friction Cone constraint (16 rows)
- 4 Unilaterality of Normal force constraint (4 rows)

} 20 constraints

Whole-Body Control: Examples



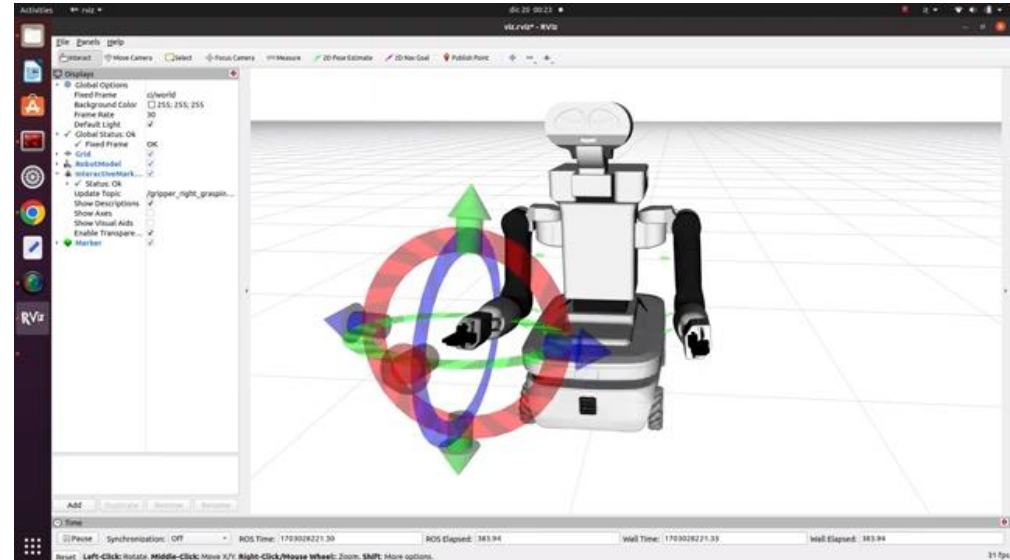
Motion planning

Motion control

reaching quadrupedal position using HIK

surface reacher stops bowing motion when contact is detected between hands and bricks

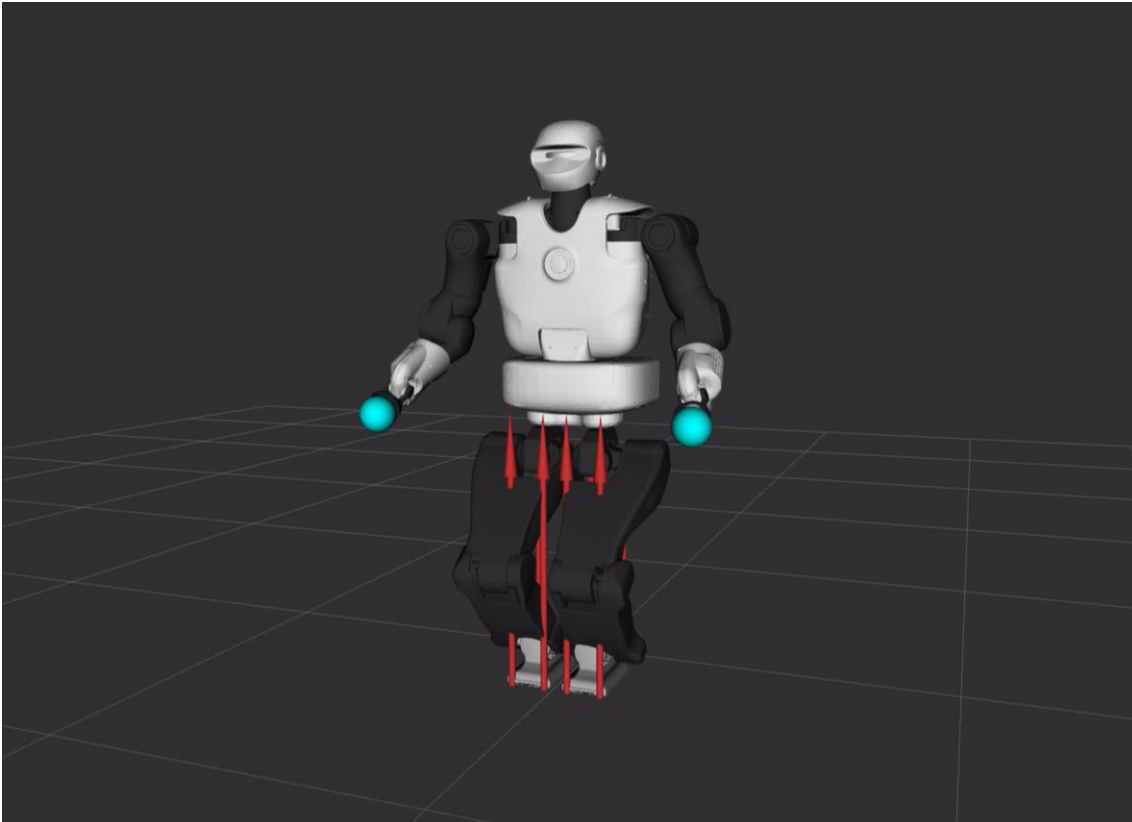
ISTITUTO ITALIANO DI TECNOLOGIA



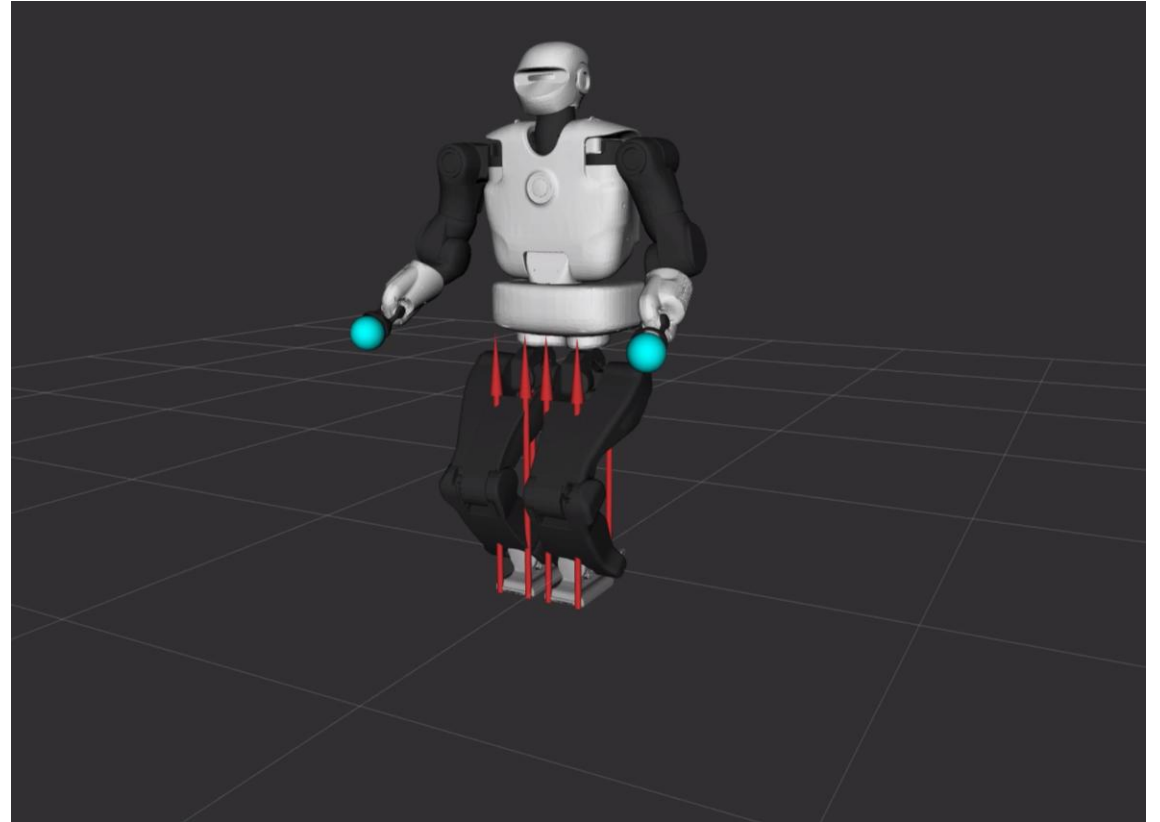
Whole-Body Control: Software

- **OpenSoT:** <https://github.com/hucebot/OpenSoT>
- **TSID:** <https://github.com/stack-of-tasks/tsid>
- **pink:** <https://github.com/stephane-caron/pink>
- **Tasks:** <https://github.com/jrl-umi3218/Tasks>

...but Whole-Body Control is not enough...



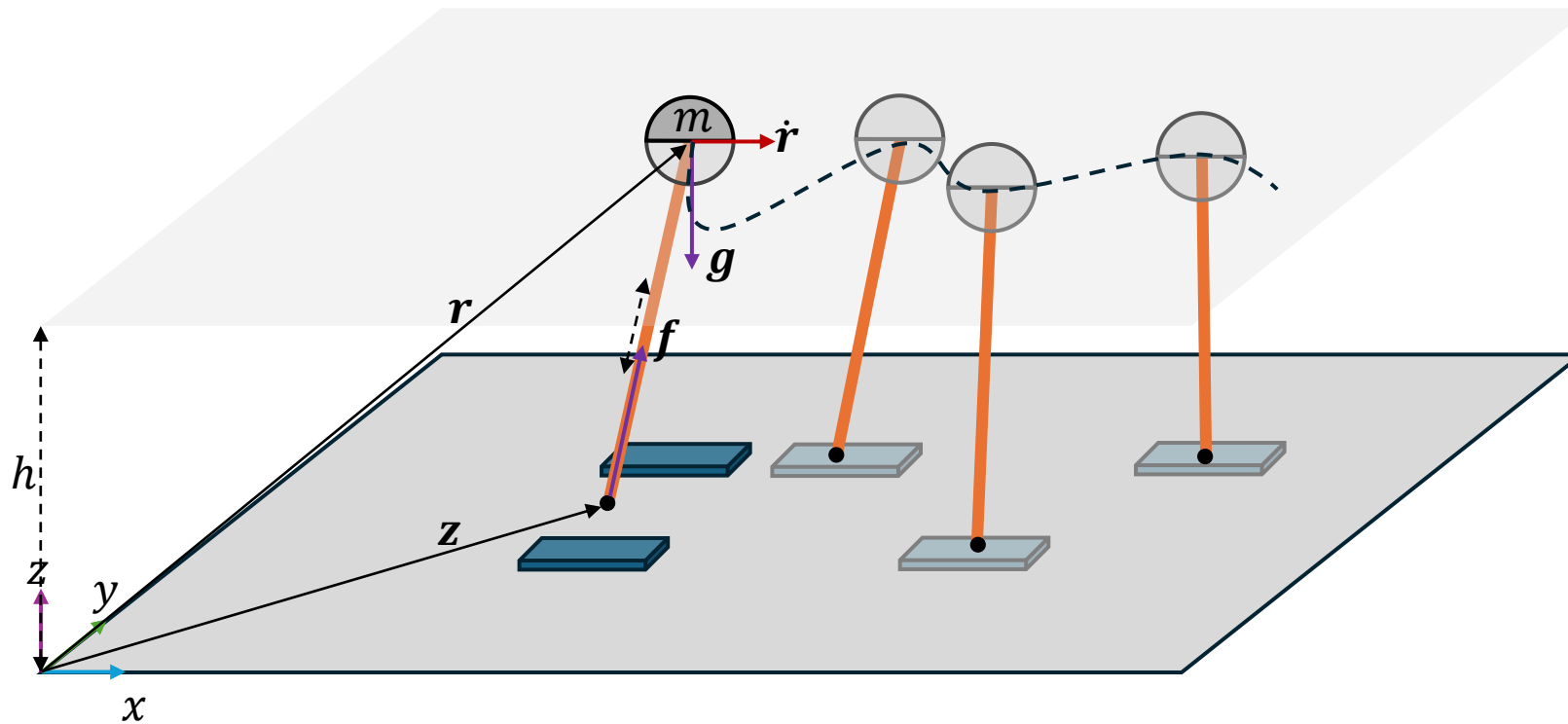
Rising the left foot w/o releasing the contact force
(dynamically inconsistent)



Rising the left foot

Linear Inverted Pendulum Model (LIPM)

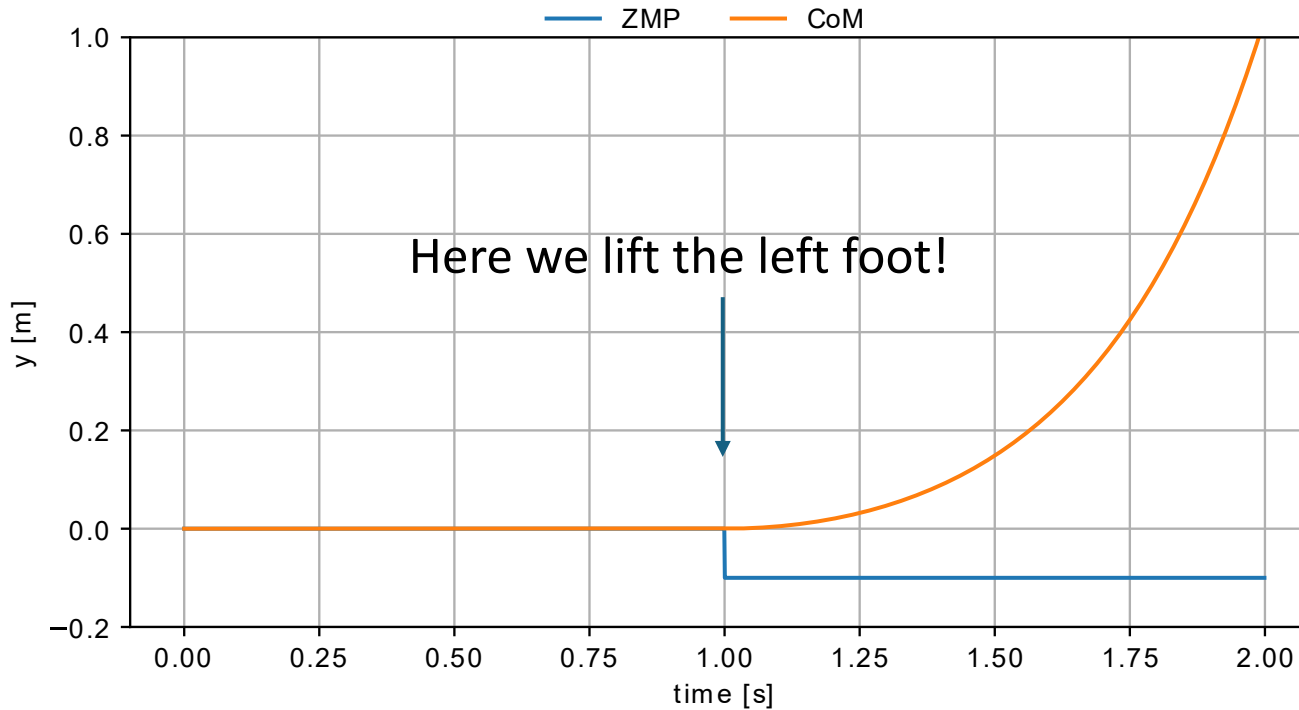
The pendulum can move only on a plane; this assumption makes it linear!



$$\begin{cases} \ddot{\mathbf{r}} = \omega^2(\mathbf{r} - \mathbf{z}) \\ \omega = \sqrt{\frac{g}{h}} \end{cases}$$

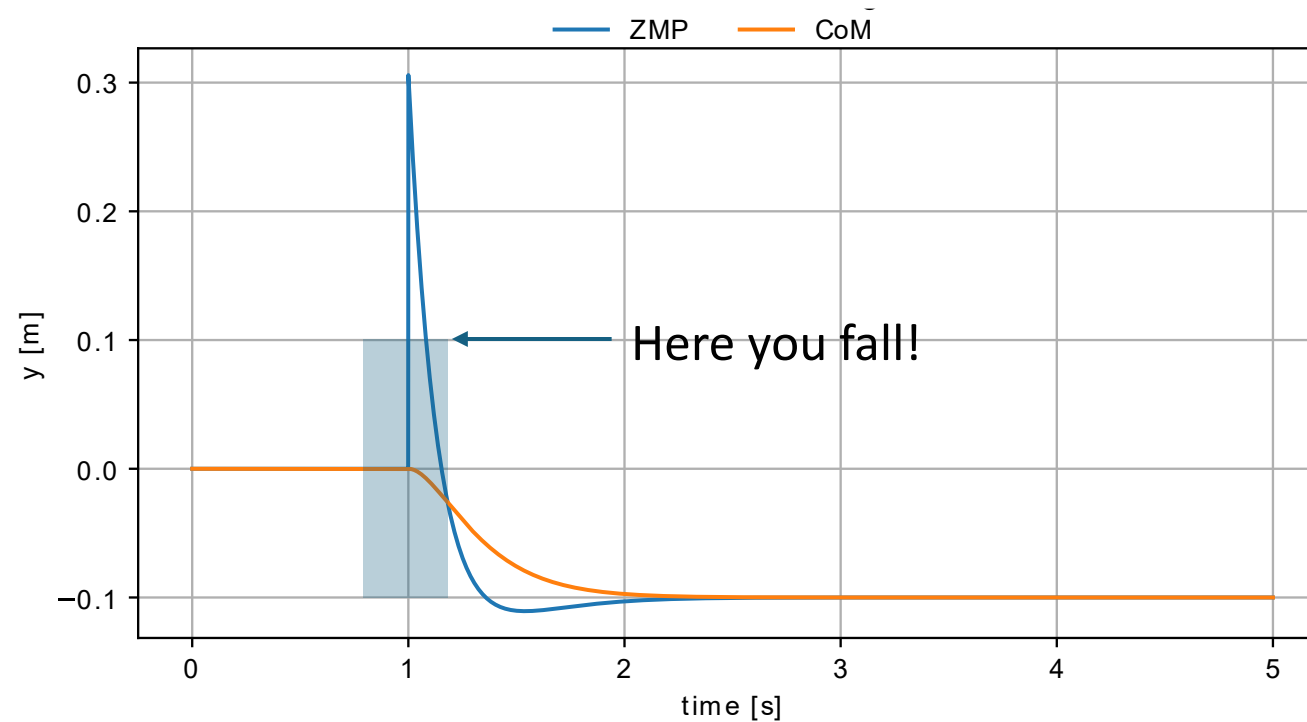
$\mathbf{z}$ is the **Zero Moment Point (ZMP)***: that point w.r.t. which reaction forces at the contacts between the feet and the ground do not produce any moment in the horizontal direction, *i.e.*, the XY .

Instantaneous ZMP motion



We need (intuitively) move the CoM **before** lifting the foot! The whole-body controller alone is not able to *predict* that.

CoM motion with PD law



Reducing the motion of the CoM makes possible to lift the foot **BUT** we go in a quasi-static locomotion regime...

Optimal Control

With Optimal Control we optimize a **trajectory** hence we consider a **preview horizon**.

Continuous form

transcription
↓

Discrete form

$$\begin{aligned} \min_{\mathbf{x}(t), \mathbf{u}(t)} & \int_0^T L(\mathbf{x}(t), \mathbf{u}(t)) dt + E(\mathbf{x}(T)) \\ \text{s. t.} & \quad \dot{\mathbf{x}} = f(\mathbf{x}(t), \mathbf{u}(t)) \\ & \quad \mathbf{c}(\mathbf{x}(t), \mathbf{u}(t)) \leq \mathbf{0} \end{aligned}$$

$$\min_{\mathbf{X}, \mathbf{U}} \sum_{k=0}^{N-1} L(\mathbf{x}_k, \mathbf{u}_k) + E(\mathbf{x}_N)$$

$$\begin{aligned} \text{s. t.} \quad & \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \quad \forall k \\ & \mathbf{c}(\mathbf{x}_k, \mathbf{u}_k) \leq \mathbf{0} \quad \forall k \end{aligned}$$

with

$$\begin{aligned} \mathbf{X} &= [\mathbf{x}_0^T \quad \mathbf{x}_1^T \quad \dots \quad \mathbf{x}_{N-1}^T \quad \mathbf{x}_N^T]^T \\ \mathbf{U} &= [\mathbf{u}_0^T \quad \mathbf{u}_1^T \quad \dots \quad \mathbf{u}_{N-1}^T]^T \end{aligned}$$

Linear Optimal Control

In **Linear** Optimal Control, costs are **Quadratic**, and constraints are **Linear** (including dynamics)

$$\min_{\mathbf{X}, \mathbf{U}} \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N + \mathbf{g}_T^T \mathbf{x}_N$$

s. t.

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k \quad \forall k$$

$$\mathbf{E} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} \leq \mathbf{e} \quad \forall k$$

$\mathbf{x}_k \in \mathbb{R}^{n_x}$	$\mathbf{X} \in \mathbb{R}^{n_x N_s}$	$\mathbf{H}_k \in \mathbb{R}^{n_x+n_u \times n_x+n_u}$	$\mathbf{A}_k \in \mathbb{R}^{n_x \times n_x}$	$\mathbf{E} \in \mathbb{R}^{m \times n_x+n_u}$
$\mathbf{u}_k \in \mathbb{R}^{n_u}$	$\mathbf{U} \in \mathbb{R}^{n_u N_{s-1}}$	$\mathbf{g}_k \in \mathbb{R}^{n_x+n_u}$	$\mathbf{B}_k \in \mathbb{R}^{n_x \times n_u}$	$\mathbf{e} \in \mathbb{R}^m$
		$\mathbf{H}_T \in \mathbb{R}^{n_x \times n_x}$		
		$\mathbf{g}_T \in \mathbb{R}^{n_x}$		

If $\mathbf{H}_k$, $\mathbf{H}_T$, $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{E}$ are not dependent on time, the system is also **Time Invariant**.

Linear Optimal Control and LIPM

We can model the LIPM as a **Linear Time Invariant (LTI)** Optimal Control problem

$$\begin{aligned} \min_{\mathbf{X}, \mathbf{U}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N + \mathbf{g}_T^T \mathbf{x}_N \\ \text{s. t.} \quad & \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k \quad \forall k \\ & \mathbf{x}_0 = \mathbf{x}_m \quad \longleftarrow \text{Initial state} \end{aligned}$$

Linear Optimal Control and LIPM

State space system:

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{r}_k \\ \dot{\mathbf{r}}_k \end{bmatrix} \quad \dot{\mathbf{x}}_k = \begin{bmatrix} \dot{\mathbf{r}}_k \\ \ddot{\mathbf{r}}_k \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{r}}_k \\ \omega^2(\mathbf{r} - \mathbf{z}) \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \omega^2 \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{x}_k + \begin{bmatrix} \mathbf{0} \\ -\omega^2 \mathbf{I} \end{bmatrix} \mathbf{u}_k$$

$\mathbf{u}_k = \mathbf{z}_k$ 2 eigen values **positive** and 2 **negative**

- $\mathbf{r}_k$: position CoM (XY)
- $\dot{\mathbf{r}}_k$: velocity CoM
- $\mathbf{z}_k$: position ZMP

Linear Optimal Control and LIPM

State space system:

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{r}_k \\ \dot{\mathbf{r}}_k \end{bmatrix} \quad \dot{\mathbf{x}}_k = \begin{bmatrix} \dot{\mathbf{r}}_k \\ \ddot{\mathbf{r}}_k \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{r}}_k \\ \omega^2(\mathbf{r} - \mathbf{z}) \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \omega^2 \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{x}_k + \begin{bmatrix} \mathbf{0} \\ -\omega^2 \mathbf{I} \end{bmatrix} \mathbf{u}_k$$

$\mathbf{u}_k = \mathbf{z}_k$

2 eigen values **positive** and 2 **negative**

(Explicit) Euler integrator:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + dt(\mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k) \longrightarrow \mathbf{x}_k + dt(\mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k) - \mathbf{x}_{k+1} = \mathbf{0}$$

$$(\mathbf{I} + dt\mathbf{A})\mathbf{x}_k + dt\mathbf{B}\mathbf{u}_k - \mathbf{x}_{k+1} = \mathbf{0}$$

$$\longrightarrow \underbrace{[\mathbf{I} + dt\mathbf{A}]}_A \underbrace{dt\mathbf{B}}_B \underbrace{[-\mathbf{I}]}_C \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \\ \mathbf{x}_{k+1} \end{bmatrix} = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\begin{cases} \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k & \forall k \\ \mathbf{x}_0 = \mathbf{x}_m \end{cases}$$



$$\begin{cases} \begin{bmatrix} \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \ddots & \dots & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \mathbf{x}_1 \\ \mathbf{u}_1 \\ \mathbf{x}_2 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{x}_{N-1} \\ \mathbf{u}_{N-1} \\ \mathbf{x}_N \end{bmatrix} = \mathbf{0} \\ \\ [\mathbf{I} \quad \mathbf{0} \quad \dots \quad \mathbf{0}] \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \mathbf{x}_m \end{cases}$$

Linear Optimal Control and LIPM

$$\begin{cases} \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k & \forall k \\ \mathbf{x}_0 = \mathbf{x}_m \end{cases}$$

$$\begin{cases} \underbrace{\begin{bmatrix} \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & \dots & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \ddots & \dots & \mathbf{I} + dt\mathbf{A} & dt\mathbf{B} & -\mathbf{I} \end{bmatrix}}_{\mathbf{E}_f} \underbrace{\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \mathbf{x}_1 \\ \mathbf{u}_1 \\ \mathbf{x}_2 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{x}_{N-1} \\ \mathbf{u}_{N-1} \\ \mathbf{x}_N \end{bmatrix}}_{\mathbf{w}} = \mathbf{0} \\ \underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{0} & \dots & \mathbf{0} \end{bmatrix}}_{\mathbf{E}_O} \underbrace{\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \vdots \\ \mathbf{x}_n \end{bmatrix}}_{\mathbf{w}} = \mathbf{x}_m \end{cases}$$

$$\begin{bmatrix} \mathbf{E}_f \\ \mathbf{E}_O \end{bmatrix} \mathbf{w} = \begin{bmatrix} \mathbf{0} \\ \mathbf{x}_m \end{bmatrix} \Rightarrow \mathbf{E}\mathbf{w} = \mathbf{e}$$

Linear Optimal Control and LIPM

$$\min_{\mathbf{X}, \mathbf{U}} \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N + \mathbf{g}_T^T \mathbf{x}_N, \quad \text{with} \quad \begin{aligned} \mathbf{x}_k &= \begin{bmatrix} \mathbf{r}_k \\ \dot{\mathbf{r}}_k \end{bmatrix} \\ \mathbf{u}_k &= \mathbf{z}_k \end{aligned}$$

Tasks to be performed:

- Track ZMP reference
- Minimize CoM velocity during tracking
- End with 0 CoM velocity

$$\Rightarrow F = \sum_{k=0}^{N-1} \left[\frac{1}{2} Q \dot{\mathbf{r}}_k^T \dot{\mathbf{r}}_k + \frac{1}{2} R (\mathbf{z}_k - \mathbf{z}_{k,d})^T (\mathbf{z}_k - \mathbf{z}_{k,d}) \right] + \frac{1}{2} Q_T \dot{\mathbf{r}}_N^T \dot{\mathbf{r}}_N$$

Linear Optimal Control and LIPM

$$\min_{X,U} \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N + \mathbf{g}_T^T \mathbf{x}_N, \quad \text{with} \quad \begin{aligned} \mathbf{x}_k &= \begin{bmatrix} \mathbf{r}_k \\ \dot{\mathbf{r}}_k \end{bmatrix} \\ \mathbf{u}_k &= \mathbf{z}_k \end{aligned}$$

Tasks to be performed:

- Track ZMP reference
- Minimize CoM velocity during tracking
- End with 0 CoM velocity

$$\begin{aligned} \Rightarrow F &= \sum_{k=0}^{N-1} \left[\frac{1}{2} Q \dot{\mathbf{r}}_k^T \dot{\mathbf{r}}_k + \frac{1}{2} R (\mathbf{z}_k - \mathbf{z}_{k,d})^T (\mathbf{z}_k - \mathbf{z}_{k,d}) \right] + \frac{1}{2} Q_T \dot{\mathbf{r}}_N^T \dot{\mathbf{r}}_N = \\ &= \frac{1}{2} \sum_{k=0}^{N-1} \left[\frac{1}{2} Q \dot{\mathbf{r}}_k^T \dot{\mathbf{r}}_k + \frac{1}{2} R \mathbf{z}_k^T \mathbf{z}_k - R \mathbf{z}_{k,d}^T \mathbf{z}_k + \cancel{\frac{1}{2} R \mathbf{z}_{k,d}^T \mathbf{z}_{k,d}} \right] + \frac{1}{2} Q_T \dot{\mathbf{r}}_N^T \dot{\mathbf{r}}_N = \end{aligned}$$

$$\Rightarrow \mathbf{H}_k = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & Q\mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & R\mathbf{I} \end{bmatrix}, \quad \mathbf{g}_k = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ -R\mathbf{z}_{k,d} \end{bmatrix}, \quad \mathbf{H}_T = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & Q_T\mathbf{I} \end{bmatrix}, \quad \mathbf{g}_T = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix},$$

Linear Optimal Control and LIPM

$$\begin{aligned}
 \min_{X,U} \sum_{k=0}^{N-1} \frac{1}{2} [\mathbf{x}_k^T \quad \mathbf{u}_k^T] \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N = \\
 = \underbrace{[\mathbf{x}_0^T \quad \mathbf{u}_0^T \quad \mathbf{x}_1^T \quad \mathbf{u}_1^T \quad \dots \quad \mathbf{x}_{N-1}^T \quad \mathbf{u}_{N-1}^T \quad \mathbf{x}_N^T]}_{\mathbf{w}^T} \underbrace{\begin{bmatrix} \mathbf{H}_0 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{H}_1 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \ddots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{H}_{N-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{H}_T \end{bmatrix}}_{\mathbf{H}} \underbrace{\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \mathbf{x}_1 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{x}_{N-1} \\ \mathbf{u}_{N-1} \\ \mathbf{x}_N \end{bmatrix}}_{\mathbf{w}} + \underbrace{[\mathbf{g}_0^T \quad \mathbf{g}_1^T \quad \dots \quad \mathbf{g}_{N-1}^T \quad \mathbf{g}_T^T]}_{\mathbf{g}^T} \underbrace{\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \\ \mathbf{x}_1 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{x}_{N-1} \\ \mathbf{u}_{N-1} \\ \mathbf{x}_N \end{bmatrix}}_{\mathbf{w}} = \\
 = \frac{1}{2} \mathbf{w}^T \mathbf{H} \mathbf{w} + \mathbf{g}^T \mathbf{w}
 \end{aligned}$$

Linear Optimal Control and LIPM

We can model the LIPM as a **Linear Time Invariant (LTI)** Optimal Control problem

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N \\ \text{s. t.} \quad & \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k \quad \forall k \\ & \mathbf{x}_0 = \mathbf{x}_m \quad \longleftarrow \text{Initial state} \end{aligned}$$


$$\begin{aligned} \min_{\mathbf{w}} \quad & \frac{1}{2} \mathbf{w}^T \mathbf{H} \mathbf{w} + \mathbf{g}^T \mathbf{w} \\ \text{s. t.} \quad & \mathbf{E} \mathbf{w} = \mathbf{e} \end{aligned} \xrightarrow{\text{solution}} \begin{array}{c} \text{KKT System} \\ \begin{bmatrix} \mathbf{H} & \mathbf{E}^T \\ \mathbf{E} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{g} \\ \mathbf{e} \end{bmatrix} \end{array}$$

Linear Optimal Control and LIPM

- Let's check dimensions: $N = 100$

Element	Symbol	Dimension
State k	$\mathbf{x}_k = \begin{bmatrix} \mathbf{r}_k \\ \dot{\mathbf{r}}_k \end{bmatrix}$	$n_x = 4$
Control k	$\mathbf{u}_k = \mathbf{z}_k$	$n_u = 2$
States	$\mathbf{X}$	$n_x \cdot N + 1 = 404$
Controls	$\mathbf{U}$	$n_u \cdot N = 400$
Hessian	$\mathbf{H}$	$(n_x + n_u) \cdot N + n_x \times (n_x + n_u) \cdot N + n_x = 604 \times 604$
Integration Constraint	$\mathbf{E}_f$	$n_x \cdot N \times (n_x + n_u) \cdot N + n_x = 400 \times 604$
Initial State Constraint	$\mathbf{E}_O$	$n_x \times (n_x + n_u) \cdot N + n_x = 4 \times 604$

Linear Optimal Control and LIPM

- Can we solve more efficiently?

For LTI:

$$\begin{bmatrix} H & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} w \\ \lambda \end{bmatrix} = \begin{bmatrix} -g \\ e \end{bmatrix}$$

↑
constant

We can compute the inverse of the coefficient matrix once and re-use it changing only the RHS term.

For NON-TI: exploit sparsity or/and structure!

Linear Optimal Control and LIPM

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \mathbf{H}_k \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \mathbf{g}_k^T \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{H}_T \mathbf{x}_N \\ \text{s. t.} \quad & \mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{u}_k \quad \forall k \\ & \mathbf{x}_0 = \mathbf{x}_m \end{aligned}$$



$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T \\ \mathbf{S}_k & \mathbf{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \begin{bmatrix} \mathbf{q}_k^T & \mathbf{r}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{Q}_T \mathbf{x}_N + \mathbf{q}_N^T \mathbf{x}_N \\ \text{s. t.} \quad & \mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \quad \forall k \\ & \mathbf{x}_0 = \mathbf{x}_m \end{aligned}$$

Linear Optimal Control and LIPM

$$\min_{\mathbf{X}, \mathbf{U}} \sum_{k=0}^{N-1} \frac{1}{2} [\mathbf{x}_k^T \quad \mathbf{u}_k^T] \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T \\ \mathbf{S}_k & \mathbf{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + [\mathbf{q}_k^T \quad \mathbf{r}_k^T] \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{Q}_T \mathbf{x}_N + \mathbf{q}_N^T \mathbf{x}_N$$

s. t. $\mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \quad \forall k$
 $\mathbf{x}_0 = \mathbf{x}_m$

- Let's consider the final part of the horizon, from node $k = N - 1$ to N :

$$\min_{\mathbf{X}, \mathbf{U}} \frac{1}{2} [\mathbf{x}_k^T \quad \mathbf{u}_k^T] \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T \\ \mathbf{S}_k & \mathbf{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + [\mathbf{q}_k^T \quad \mathbf{r}_k^T] \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{P}_T \mathbf{x}_N + \mathbf{p}_N^T \mathbf{x}_N$$

s. t. $\mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k$

with $\mathbf{P}_N = \mathbf{Q}_T$ and $\mathbf{p}_N = \mathbf{q}_N$.

Linear Optimal Control and LIPM

$$\min_{\mathbf{x}, \mathbf{u}} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T \\ \mathbf{S}_k & \mathbf{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \begin{bmatrix} \mathbf{q}_k^T & \mathbf{r}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{P}_T \mathbf{x}_N + \mathbf{p}_N^T \mathbf{x}_N$$

$$\text{s. t. } \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k$$

The *Lagrangian* function of this problem has the following form:

$$\mathcal{L} = \frac{1}{2} \mathbf{x}_k^T \mathbf{Q}_k \mathbf{x}_k + \frac{1}{2} \mathbf{u}_k^T \mathbf{R}_k \mathbf{u}_k + \mathbf{x}_k^T \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k^T \mathbf{x}_k + \mathbf{r}_k^T \mathbf{u}_k + \frac{1}{2} \mathbf{x}_N^T \mathbf{P}_T \mathbf{x}_N + \mathbf{p}_N^T \mathbf{x}_N + \boldsymbol{\pi}_N^T (\mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k)$$

Linear Optimal Control and LIPM

$$\min_{\mathbf{x}, \mathbf{u}} \frac{1}{2} \begin{bmatrix} \mathbf{x}_k^T & \mathbf{u}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T \\ \mathbf{S}_k & \mathbf{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \begin{bmatrix} \mathbf{q}_k^T & \mathbf{r}_k^T \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} + \frac{1}{2} \mathbf{x}_N^T \mathbf{P}_T \mathbf{x}_N + \mathbf{p}_N^T \mathbf{x}_N$$

$$\text{s. t. } \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k$$

The *Lagrangian* function of this problem has the following form:

$$\mathcal{L} = \frac{1}{2} \mathbf{x}_k^T \mathbf{Q}_k \mathbf{x}_k + \frac{1}{2} \mathbf{u}_k^T \mathbf{R}_k \mathbf{u}_k + \mathbf{x}_k^T \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k^T \mathbf{x}_k + \mathbf{r}_k^T \mathbf{u}_k + \frac{1}{2} \mathbf{x}_N^T \mathbf{P}_T \mathbf{x}_N + \mathbf{p}_N^T \mathbf{x}_N + \boldsymbol{\pi}_N^T (\mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k)$$

we take derivatives to form the KKT system:

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \end{array} \right. \quad \longleftrightarrow \quad \begin{bmatrix} \mathbf{Q}_k & \mathbf{S}_k^T & \mathbf{0} & \mathbf{A}_k^T \\ \mathbf{S}_k & \mathbf{R}_k & \mathbf{0} & \mathbf{B}_k^T \\ \mathbf{0} & \mathbf{0} & \mathbf{P}_N & -\mathbf{I} \\ \mathbf{A}_k & \mathbf{B}_k & -\mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \\ \mathbf{x}_N \\ \boldsymbol{\pi}_N \end{bmatrix} = \begin{bmatrix} -\mathbf{q}_k \\ -\mathbf{r}_k \\ -\mathbf{p}_N \\ -\mathbf{b}_k \end{bmatrix}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ①} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ②} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \text{ ③} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \text{ ④} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{1} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{2} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{3} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \quad \textcircled{4} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

$$\Rightarrow \mathbf{P}_N (\mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k) + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ①} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ②} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \text{ ③} \longrightarrow \boldsymbol{\pi}_N = \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \text{ ④} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

$$\Rightarrow \mathbf{P}_N (\mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k) + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0}$$

$$\Rightarrow \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ①} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ ②} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \text{ ③} \longrightarrow \boldsymbol{\pi}_N = \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \text{ ④} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

$$\Rightarrow \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T (\mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N) = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{1} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{2} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \quad \textcircled{3} \longrightarrow \boldsymbol{\pi}_N = \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \quad \textcircled{4} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

$$\Rightarrow \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T (\mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N) = \mathbf{0}$$

$$\Rightarrow \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{b}_k + \mathbf{A}_k^T \mathbf{p}_N = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ (1)} \longrightarrow \bar{\mathbf{Q}}_k \mathbf{x}_k + \bar{\mathbf{S}}_k^T \mathbf{u}_k + \bar{\mathbf{q}}_k = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ (2)} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \text{ (3)} \longrightarrow \boldsymbol{\pi}_N = \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \text{ (4)} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right.$$

$$\Rightarrow \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T (\mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N) = \mathbf{0}$$

$$\Rightarrow \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{b}_k + \mathbf{A}_k^T \mathbf{p}_N = \mathbf{0}$$

$$\Rightarrow \underbrace{(\mathbf{Q}_k + \mathbf{A}_k^T \mathbf{P}_N \mathbf{A}_k)}_{\bar{\mathbf{Q}}_k} \mathbf{x}_k + \underbrace{(\mathbf{S}_k^T + \mathbf{A}_k^T \mathbf{P}_N \mathbf{B}_k)}_{\bar{\mathbf{S}}_k^T} \mathbf{u}_k + \underbrace{[\mathbf{q}_k + \mathbf{A}_k^T (\mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N)]}_{\bar{\mathbf{q}}_k} = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k} = \mathbf{Q}_k \mathbf{x}_k + \mathbf{S}_k^T \mathbf{u}_k + \mathbf{q}_k + \mathbf{A}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ (1)} \longrightarrow \bar{\mathbf{Q}}_k \mathbf{x}_k + \bar{\mathbf{S}}_k^T \mathbf{u}_k + \bar{\mathbf{q}}_k = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{u}_k} = \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \boldsymbol{\pi}_N = \mathbf{0} \text{ (2)} \longrightarrow \bar{\mathbf{S}}_k \mathbf{x}_k + \bar{\mathbf{R}}_k \mathbf{u}_k + \bar{\mathbf{r}}_k = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}_N} = \mathbf{P}_N \mathbf{x}_N + \mathbf{p}_N - \boldsymbol{\pi}_N = \mathbf{0} \text{ (3)} \longrightarrow \boldsymbol{\pi}_N = \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\pi}_N} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k - \mathbf{x}_N + \mathbf{b}_k = \mathbf{0} \text{ (4)} \longrightarrow \mathbf{x}_N = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k \end{array} \right. \longrightarrow \begin{bmatrix} \bar{\mathbf{Q}}_k & \bar{\mathbf{S}}_k^T \\ \bar{\mathbf{S}}_k & \bar{\mathbf{R}}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} = \begin{bmatrix} -\bar{\mathbf{q}}_k \\ -\bar{\mathbf{r}}_k \end{bmatrix}$$

$$\Rightarrow \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T (\mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N) = \mathbf{0}$$

$$\Rightarrow \mathbf{R}_k \mathbf{u}_k + \mathbf{S}_k \mathbf{x}_k + \mathbf{r}_k + \mathbf{B}_k^T \mathbf{P}_N \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k^T \mathbf{P}_N \mathbf{B}_k \mathbf{u}_k + \mathbf{B}_k^T \mathbf{P}_N \mathbf{b}_k + \mathbf{B}_k^T \mathbf{p}_N = \mathbf{0}$$

$$\Rightarrow \underbrace{(\mathbf{R}_k + \mathbf{B}_k^T \mathbf{P}_N \mathbf{B}_k)}_{\bar{\mathbf{R}}_k} \mathbf{u}_k + \underbrace{(\mathbf{S}_k + \mathbf{B}_k^T \mathbf{P}_N \mathbf{A}_k)}_{\bar{\mathbf{S}}_k} \mathbf{x}_k + \underbrace{[\mathbf{r}_k + \mathbf{B}_k^T (\mathbf{P}_N \mathbf{b}_k + \mathbf{p}_N)]}_{\bar{\mathbf{r}}_k} = \mathbf{0}$$

Linear Optimal Control and LIPM

$$\begin{bmatrix} \bar{Q}_k & \bar{S}_k^T \\ \bar{S}_k & \bar{R}_k \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} = \begin{bmatrix} -\bar{q}_k \\ -\bar{r}_k \end{bmatrix}$$

Let's now assume fixed state (or state as a given parameter), we can compute the control:

$$\mathbf{u}_k = -\bar{R}_k^{-1} \bar{S}_k \mathbf{x}_k - \bar{R}_k^{-1} \bar{r}_k = \mathbf{K}_k \mathbf{x}_k + \mathbf{k}_k$$

Riccati gain feed-back and
feed-forward

Linear Optimal Control and LIPM

$$\begin{bmatrix} \bar{Q}_k & \bar{S}_k^T \\ \bar{S}_k & \bar{R}_k \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix} = \begin{bmatrix} -\bar{q}_k \\ -\bar{r}_k \end{bmatrix}$$

Let's now assume fixed state (or state as a given parameter), we can compute the control:

$$u_k = -\bar{R}_k^{-1} \bar{S}_k x_k - \bar{R}_k^{-1} \bar{r}_k = \underbrace{K_k}_{\text{Riccati gain feed-back}} x_k + \underbrace{k_k}_{\text{Riccati gain feed-forward}}$$

This expression can be used in the first row:

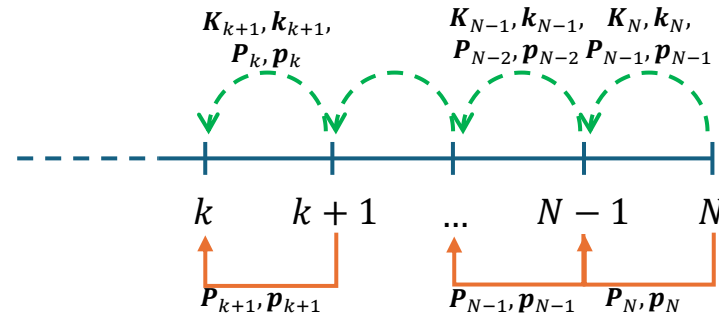
$$\bar{Q}_k x_k + \bar{S}_k^T (-\bar{R}_k^{-1} \bar{S}_k x_k - \bar{R}_k^{-1} \bar{r}_k) = -\bar{q}_k$$

$$\Rightarrow \underbrace{(\bar{Q}_k - \bar{S}_k^T \bar{R}_k^{-1} \bar{S}_k)}_{\bar{P}_k} x_k + \underbrace{\bar{q}_k - \bar{S}_k^T \bar{R}_k^{-1} \bar{r}_k}_{\bar{p}_k} = \pi_k$$

Stage-wise resolution

Recursion: at iteration k , we consider the triplet $\mathbf{x}_k$, $\mathbf{u}_k$, and $\mathbf{x}_{k+1}$

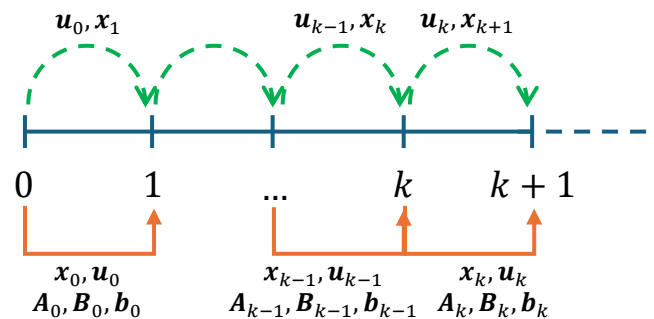
1. During the *backward pass* we compute $\bar{\mathbf{P}}_{k-1}$, $\bar{\mathbf{p}}_{k-1}$, $\mathbf{K}_k$, and $\mathbf{k}_k$, inputting $\bar{\mathbf{P}}_k$ and $\bar{\mathbf{p}}_k$ computed at the previous iteration. The *backward pass* is initialized with $\mathbf{P}_N = \mathbf{Q}_N$ and $\mathbf{p}_N = \mathbf{q}_N$, and computes backward until node 0.



Stage-wise resolution

Recursion: at iteration k , we consider the triplet $\mathbf{x}_k$, $\mathbf{u}_k$, and $\mathbf{x}_{k+1}$

1. During the *backward pass* we compute $\bar{\mathbf{P}}_k$, $\bar{\mathbf{p}}_k$, $\mathbf{K}_k$, and $\mathbf{k}_k$, inputting $\bar{\mathbf{P}}_{k+1}$ and $\bar{\mathbf{p}}_{k+1}$ computed at the previous iteration. The *backward pass* is initialized with $\mathbf{P}_N = \mathbf{Q}_N$ and $\mathbf{p}_N = \mathbf{q}_N$, and computes backward until node 0.
2. In the *forward pass*, starting from node 0 to node N , we compute the control $\mathbf{u}_k = \mathbf{K}_k \mathbf{x}_k + \mathbf{k}_k$ and the next state $\mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k$. The constraint on the initial state is given as initialization, i.e. we start to integrate from $\mathbf{x}_m$.



Stage-wise resolution

Recursion: at iteration k , we consider the triplet $\mathbf{x}_k$, $\mathbf{u}_k$, and $\mathbf{x}_{k+1}$

1. During the *backward pass* we compute $\bar{\mathbf{P}}_k$, $\bar{\mathbf{p}}_k$, $\mathbf{K}_k$, and $\mathbf{k}_k$, inputting $\bar{\mathbf{P}}_{k+1}$ and $\bar{\mathbf{p}}_{k+1}$ computed at the previous iteration. The *backward pass* is initialized with $\mathbf{P}_N = \mathbf{Q}_N$ and $\mathbf{p}_N = \mathbf{q}_N$, and computes backward until node 0.
2. In the *forward pass*, starting from node 0 to node N , we compute the control $\mathbf{u}_k = \mathbf{K}_k \mathbf{x}_k + \mathbf{k}_k$ and the next state $\mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{b}_k$. The constraint on the initial state is given as initialization, i.e. we start to integrate from $\mathbf{x}_m$.



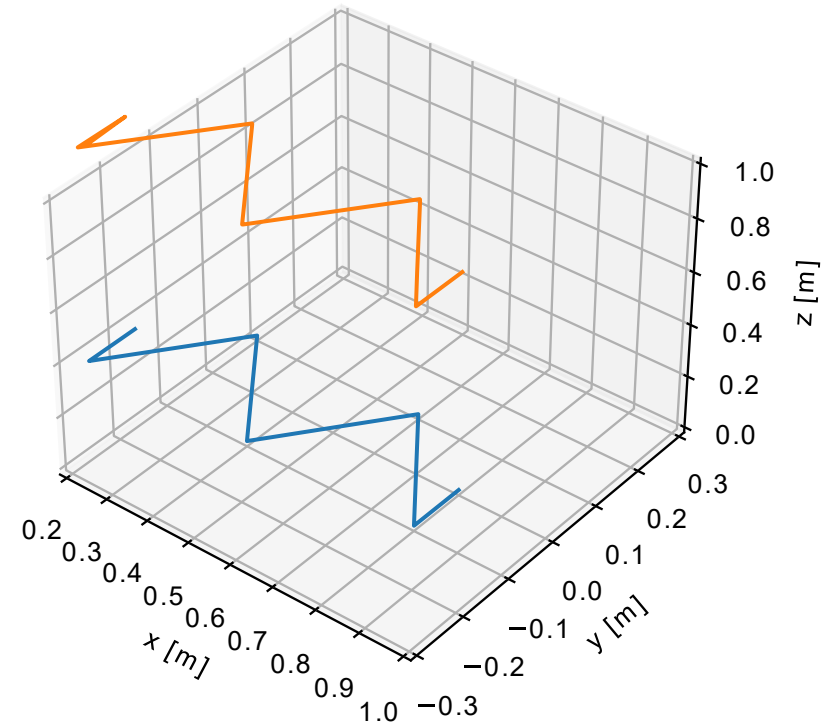
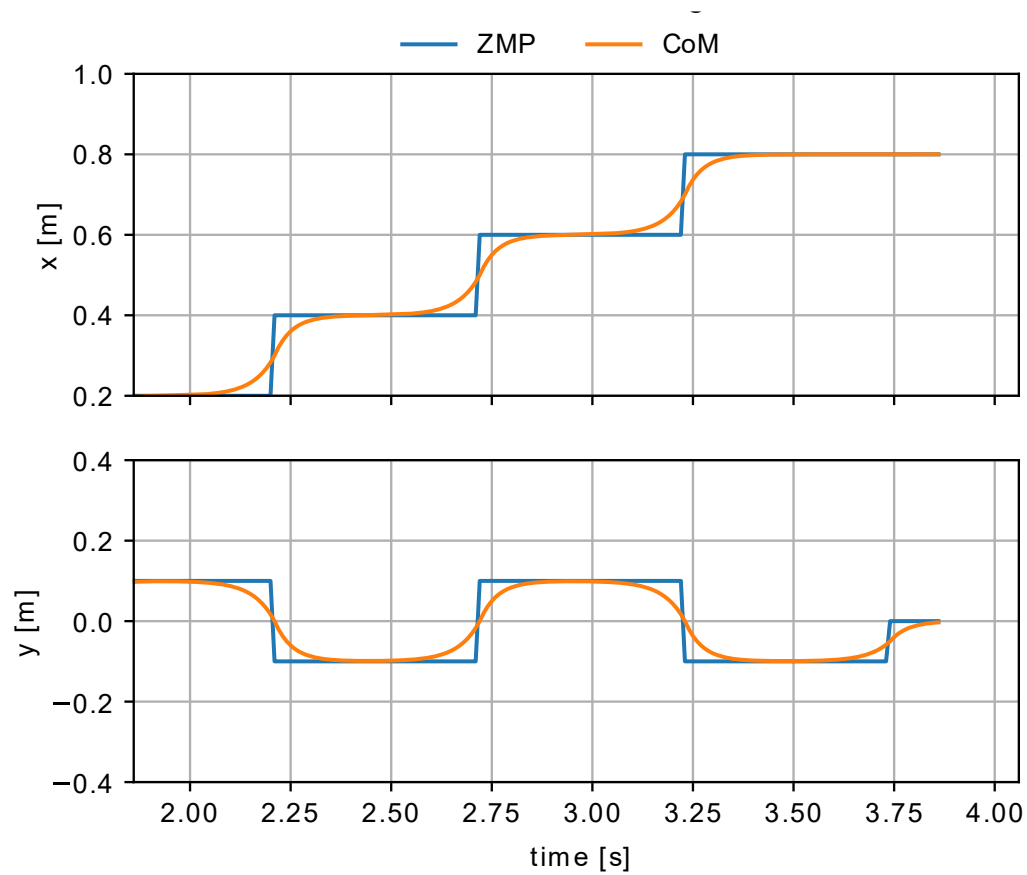
We have reduced the resolution of the problem into an inverse of the $\bar{\mathbf{R}}_k$ matrix $\mathbf{u}_n \times \mathbf{u}_n$...for N times! **Riccati recursion exploits the temporal structure of the OCP, reducing the solution complexity from cubic in the horizon length for a dense KKT solve to linear in the horizon length.**

There exists methods to incorporate constraints in stage-wise methods [1,2].

[1] Frison, Gianluca, and Moritz Diehl. "HPIPM: a high-performance quadratic programming framework for model predictive control." *IFAC-PapersOnLine* 53.2 (2020): 6563-6569.

[2] Jordana, Armand, et al. "Structure-exploiting sequential quadratic programming for model-predictive control." *IEEE Transactions on Robotics* (2025).

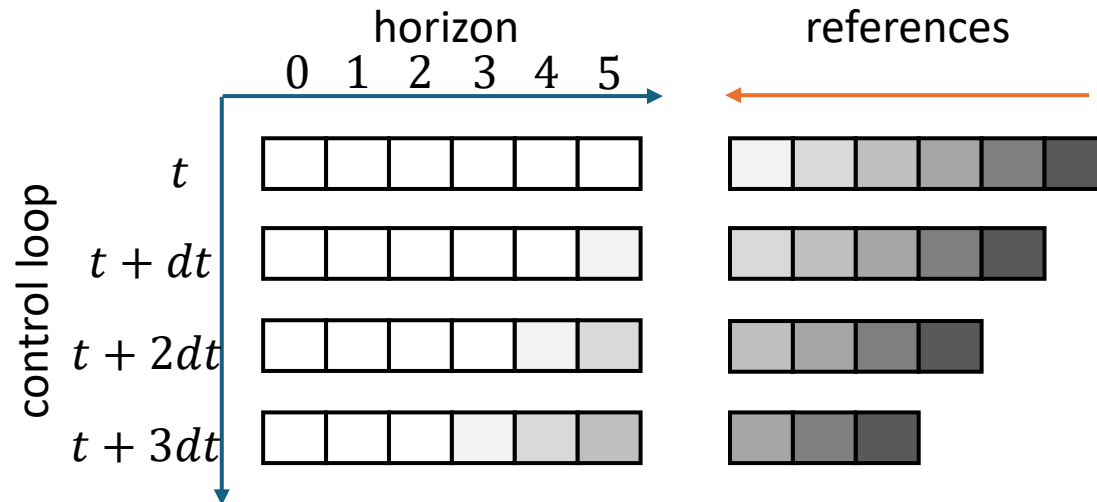
Linear Optimal Control and LIPM



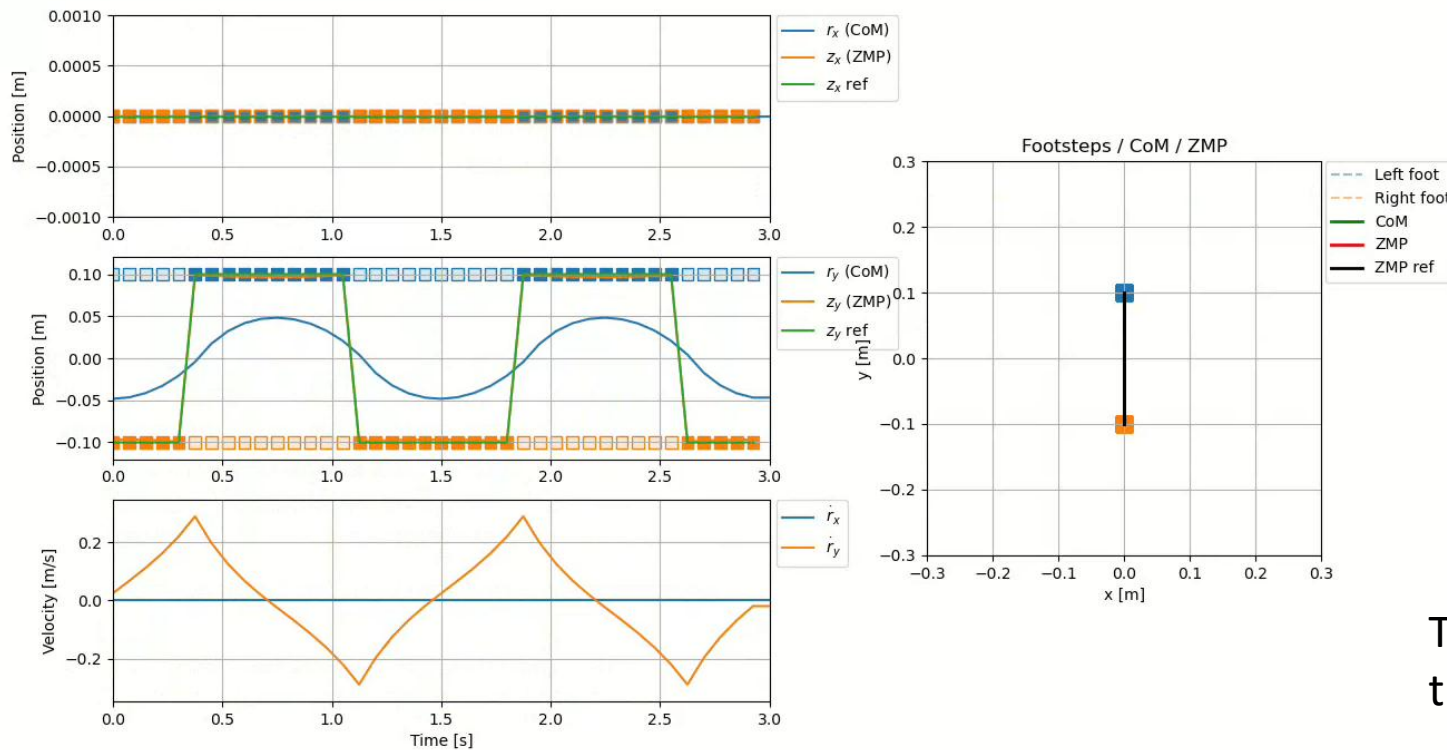
Stable!

TO vs MPC

- **TO:** single optimization (mostly used as a “*planning step*”)
 - we are interested in global optimality (multiple iterations);
 - we are not interested in on-line execution;
- **MPC:** in a control loop, at each loop is called the optimization:
 - the measured state is imposed as initial state;
 - given the optimized states and controls, **only the first control is used (+ next planned state possibly)**;
 - constraints and references “*shift to the previous node*” (Receding Horizon):



LIPM MPC (Open loop)



Nominal foot displacement

- Footstep target Law [1]:

$$\mathbf{p}_{k+1}^R = \mathbf{p}_k^L + R(\theta_k^L) \left(\mathbf{p}_0^{R/L} + \boxed{\dot{\mathbf{r}}_d T} \right)$$

a velocity-integrated step displacement, expressed in the support-foot frame.

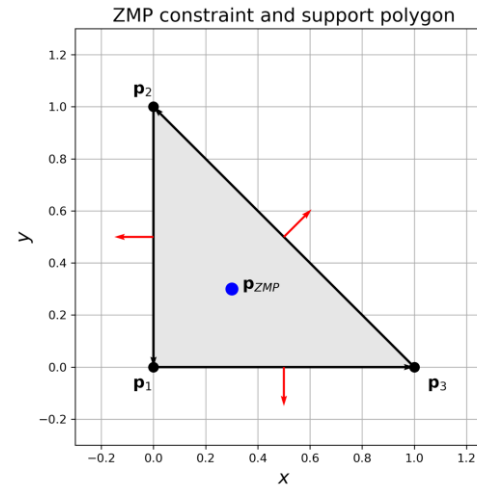
Velocity driven

The ZMP desired is then set according to the footstep target.

Stability Constraint in LIPM

We can constraint the ZMP to lie in the support polygon for each node k :

$$\left[(\mathbf{p}_{j(i),k} - \mathbf{p}_{i,k}) \times (\mathbf{z}_k - \mathbf{p}_{i,k}) \right]_z \leq 0, \quad \forall k, \forall i$$



with $\mathbf{p}_{i,k}$ a foot or a vertex of the support polygon and $\mathbf{p}_{j(i),k}$ the subsequent in clockwise ordering.

IFF the **support polygon is not a variable** of the problem, then this constraint is linear. In this case is necessary to know the pattern of the footsteps in advance, as in the previous case.

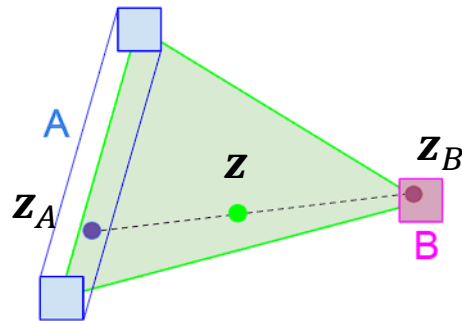
However, if the feet locations (i.e. the support polygon) are variables, this constraint is quadratic and non-convex!

Stability Constraint in LIPM

- Automatic footstep placement [1]: The ZMP must lie within the support polygon defined by the footsteps. Considering **only the single support** makes this constraint linear:

New variable(s) foot position $\rightarrow z - p_i \leq -\delta p \rightarrow$ **Side between consecutive points in support polygon**

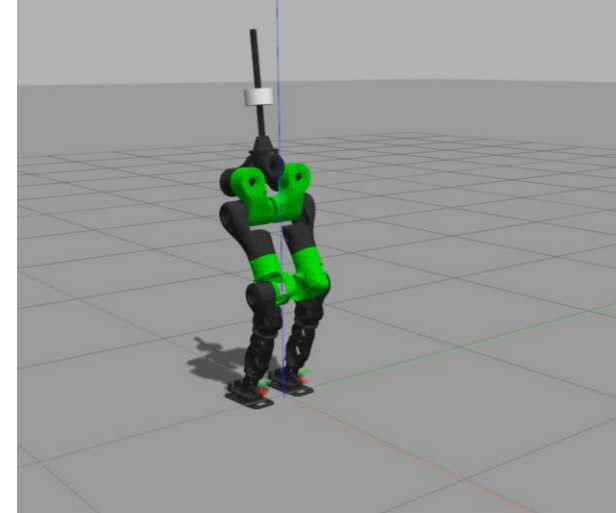
- Automatic footstep placement in quadrupeds [2]: Constraint is linearized by modeling the quadruped as 2 bipeds:



$$\alpha \mathbf{u}_A + (1 - \alpha) \mathbf{u}_B = \mathbf{u}$$

$$\alpha \mathbf{x}_A + (1 - \alpha) \mathbf{x}_B = \mathbf{x}$$

$$\alpha \in (0,1)$$

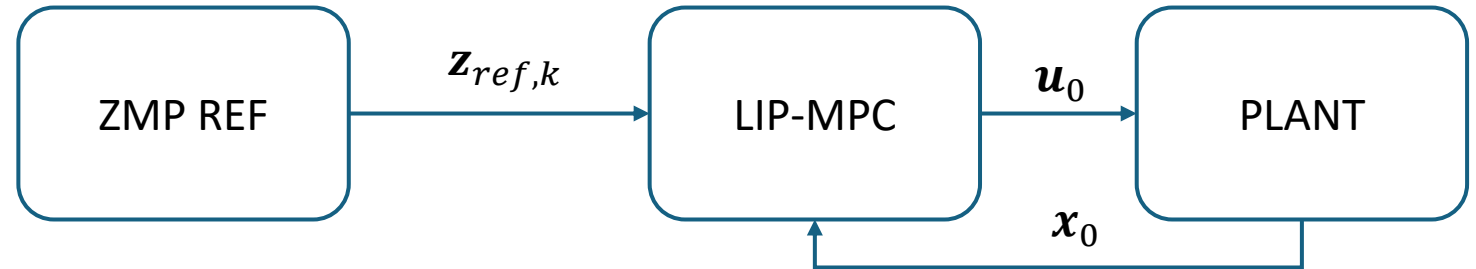


[1] Herdt, Andrei, Holger Diedam, Pierre-Brice Wieber, Dimitar Dimitrov, Katja Mombaur, and Moritz Diehl. "Online walking motion generation with automatic footstep placement." *Advanced Robotics* 24, no. 5-6 (2010): 719-737.

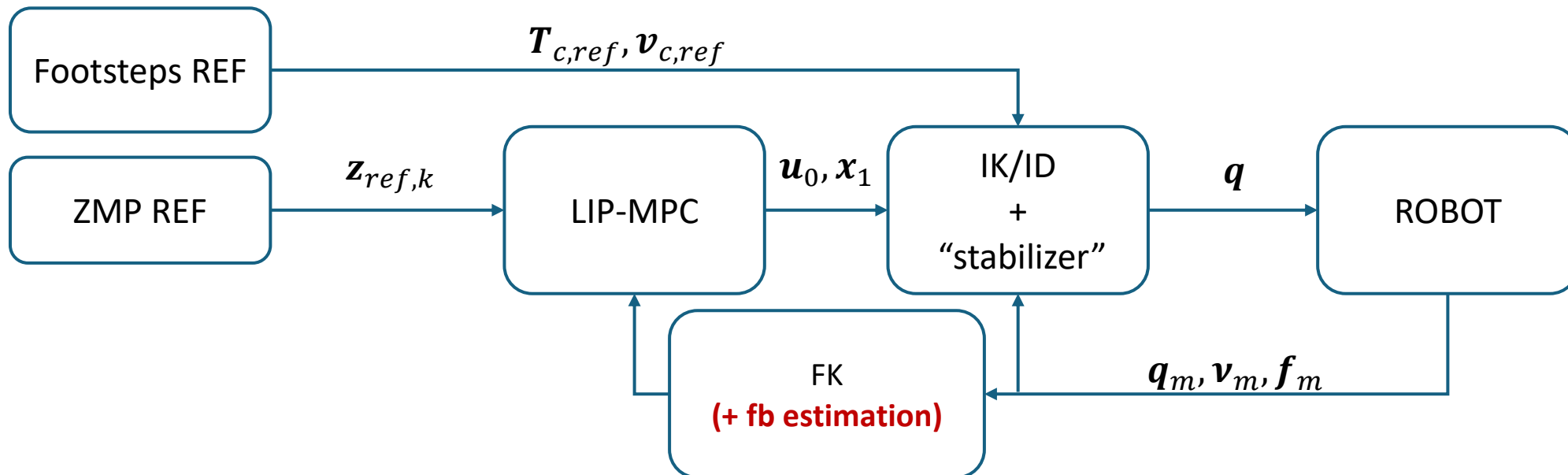
[2] Laurenzi, Arturo, Enrico Mingo Hoffman, and Nikos G. Tsagarakis. "Quadrupedal walking motion and footstep placement through linear model predictive control." *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018.

Linear Optimal Control and LIPM

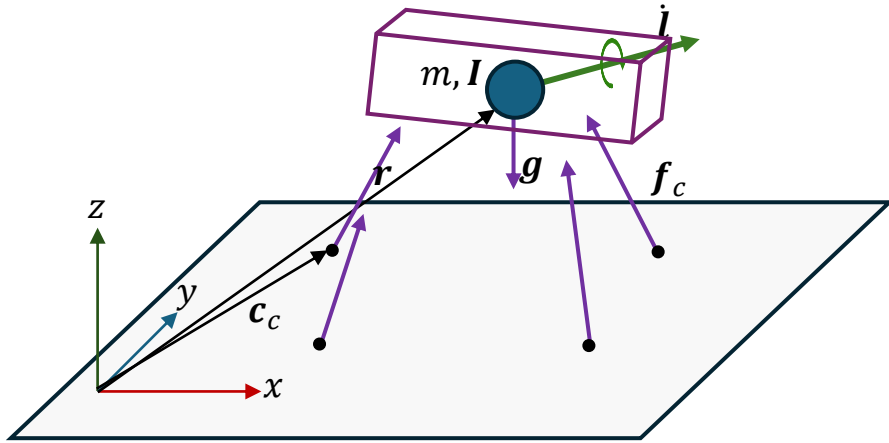
From “Control” point of view:



From “Robotics” point of view:



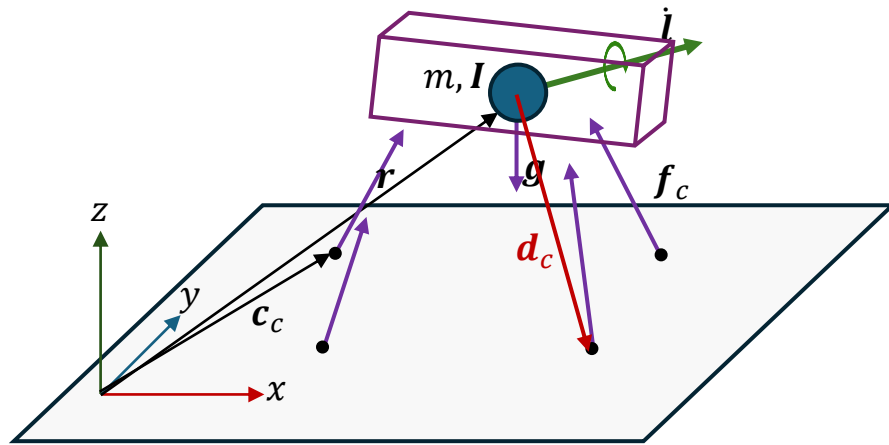
Single Rigid Body Dynamics (SRBD) Model



$$\left\{ \begin{array}{l} \dot{\mathbf{h}} = \begin{bmatrix} m\ddot{\mathbf{r}} \\ \mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \sum_{c=0}^{C-1} \mathbf{f}_c + m\mathbf{g} \\ \sum_{c=0}^{C-1} (\mathbf{c}_c - \mathbf{r}) \times \mathbf{f}_c \end{bmatrix} \\ \mathbf{I} = \mathbf{R}_B \mathbf{I}_B \mathbf{R}_B^T \end{array} \right.$$

The SRBD model is NOT linear!

Linearized SRBD Model [1-3]



$$\left\{ \begin{array}{l} \dot{h} = \left[I\dot{\omega} + \cancel{\omega \times I\omega} \right] = \begin{bmatrix} \sum_{c=0}^{C-1} f_c + mg \\ \sum_{c=0}^{C-1} \underbrace{(c_c - r)}_{d_c} \times f_c \end{bmatrix} \\ I = R_{\theta} I_B R_{\theta}^T \end{array} \right.$$

- $\omega \times I\omega \approx 0$
- d_c **is not optimized**, a value for each node is provided, e.g. from trajectory optimization (similarly as LIPM)
- Euler angles are used to parameterize the orientation of the base:

$$\theta_B = \begin{bmatrix} \phi \\ \theta \\ \varphi \end{bmatrix} \quad \dot{\theta}_B = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\varphi} \end{bmatrix} \approx R_{\theta} \omega, \text{ with } R_{\theta} \text{ provided for each node (as } d_c \text{).}$$

[1] M. Chignoli, D. Kim, E. Stanger-Jones, and S. Kim, "The mit humanoid robot: Design, motion planning, and control for acrobatic behaviors," arXiv preprint arXiv:2104.09025, 2021.

[2] D. Kim, J. Di Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," arXiv preprint arXiv:1909.06586, 2019.

[3] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the mit cheetah 3 through convex model-predictive control," in IEEE RSJ International Conference on Intelligent Robots and Systems (IROS), 2018, pp. 1–9

Linear Optimal Control and SRBD Model

State space system:

$$\mathbf{x} = \begin{bmatrix} \boldsymbol{\theta}_B \\ \mathbf{r} \\ \boldsymbol{\omega} \\ \dot{\mathbf{r}} \end{bmatrix} \in \mathbb{R}^{12}$$

$$\mathbf{u} = \begin{bmatrix} \mathbf{f}_0 \\ \vdots \\ \mathbf{f}_{C-1} \end{bmatrix} \in \mathbb{R}^{3C}$$

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{b} = \begin{matrix} \mathbb{R}^{12 \times 12} \\ \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{R}_{\boldsymbol{\theta}} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{1}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \end{matrix} \mathbf{x} + \begin{matrix} \mathbb{R}^{12 \times 3C} \\ \begin{bmatrix} \mathbf{0}_{3 \times 3} & \cdots & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \cdots & \mathbf{0}_{3 \times 3} \\ \mathbf{I}^{-1}[\mathbf{d}_0]_{\times} & \cdots & \mathbf{I}^{-1}[\mathbf{d}_{C-1}]_{\times} \\ \frac{1}{m} \mathbf{1}_{3 \times 3} & \cdots & \frac{1}{m} \mathbf{1}_{3 \times 3} \end{bmatrix} \end{matrix} \mathbf{u} + \begin{bmatrix} \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} \\ \mathbf{g} \end{bmatrix}$$

Time variant!

Euler integrator:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + dt(\mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k) + dt \mathbf{b}$$

Other constraints:

$$\begin{bmatrix} \mathbf{c}_{0,m} \\ \mathbf{c}_{1,m} \\ \vdots \\ \mathbf{c}_{C-1,m} \end{bmatrix} \leq \begin{bmatrix} \mathbf{C}_0 \mathbf{R}_0 & \mathbf{0}_{5 \times 3} & \cdots & \mathbf{0}_{5 \times 3} \\ \mathbf{0}_{5 \times 3} & \mathbf{C}_1 \mathbf{R}_1 & \cdots & \mathbf{0}_{5 \times 3} \\ & & \ddots & \\ \mathbf{0}_{5 \times 3} & \mathbf{0}_{5 \times 3} & \cdots & \mathbf{C}_{C-1} \mathbf{R}_{C-1} \end{bmatrix} \mathbf{u} \leq \begin{bmatrix} \mathbf{c}_{0,M} \\ \mathbf{c}_{1,M} \\ \vdots \\ \mathbf{c}_{C-1,M} \end{bmatrix}$$



$$\mathbf{C}_c = \begin{bmatrix} 1 & 0 & -\mu \\ -1 & 0 & -\mu \\ 0 & 1 & -\mu \\ 0 & -1 & -\mu \\ \boxed{0} & \boxed{0} & \boxed{-1} \end{bmatrix}$$

Friciton cones

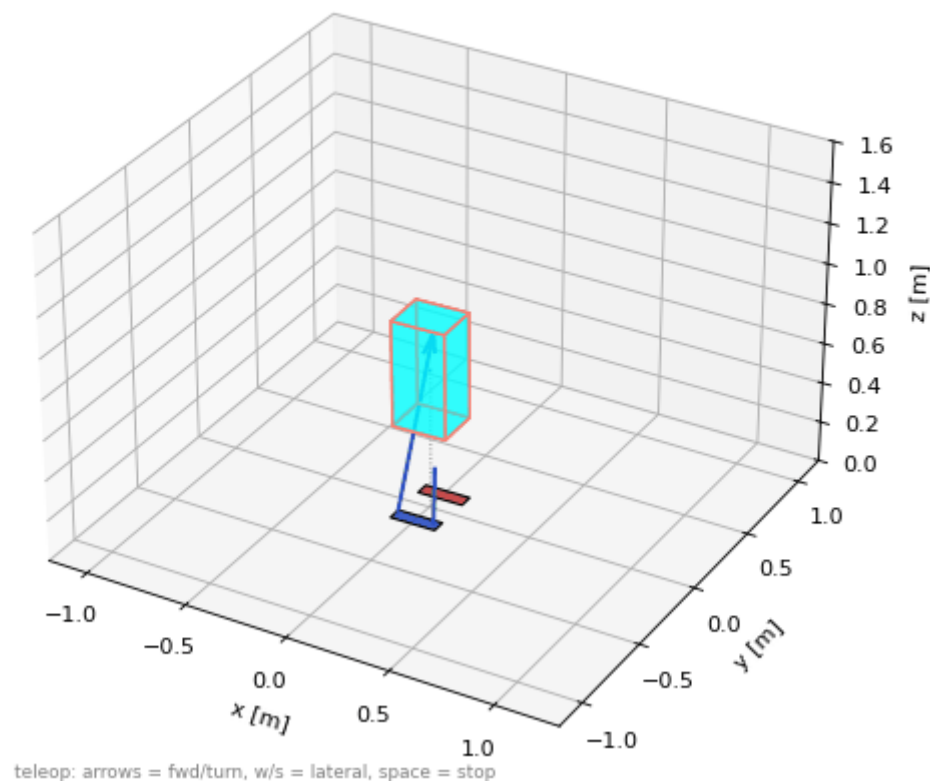
Unilaterality

$$\mathbf{c}_{c,M} = \mathbf{0}_{5 \times 1}$$

$$\mathbf{c}_{c,m} = -\infty_{5 \times 1}$$

L-SRBD MPC

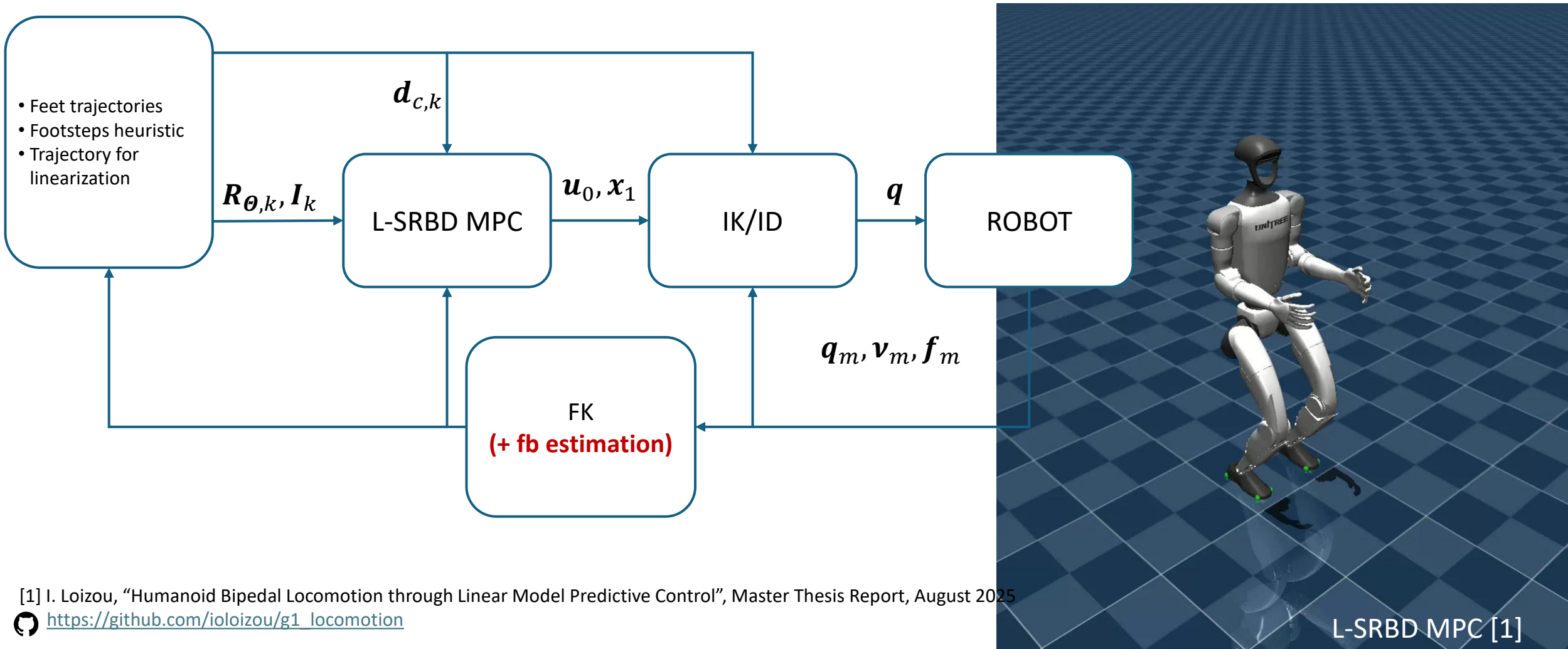
```
t = 0.08 s [walking]
cmd: fwd +0.40 lat +0.00 m/s yaw +0.2 rad/s
act: fwd +0.10 lat +0.15 m/s z 0.69 m
```



- Footstep target Law [1]:

$$\underbrace{\mathbf{p}_{k+1}^R = \mathbf{r}_k + R(\psi) \begin{bmatrix} 0 \\ -d_{hip} \\ 0 \end{bmatrix}}_{\mathbf{p}_{hip}, \text{ position of the hip of the swing foot}} + \frac{T_s}{2} \dot{\mathbf{r}} + \sqrt{\frac{h}{g}} (\dot{\mathbf{r}} - \dot{\mathbf{r}}_d)$$

Linear Optimal Control and SRBD Model



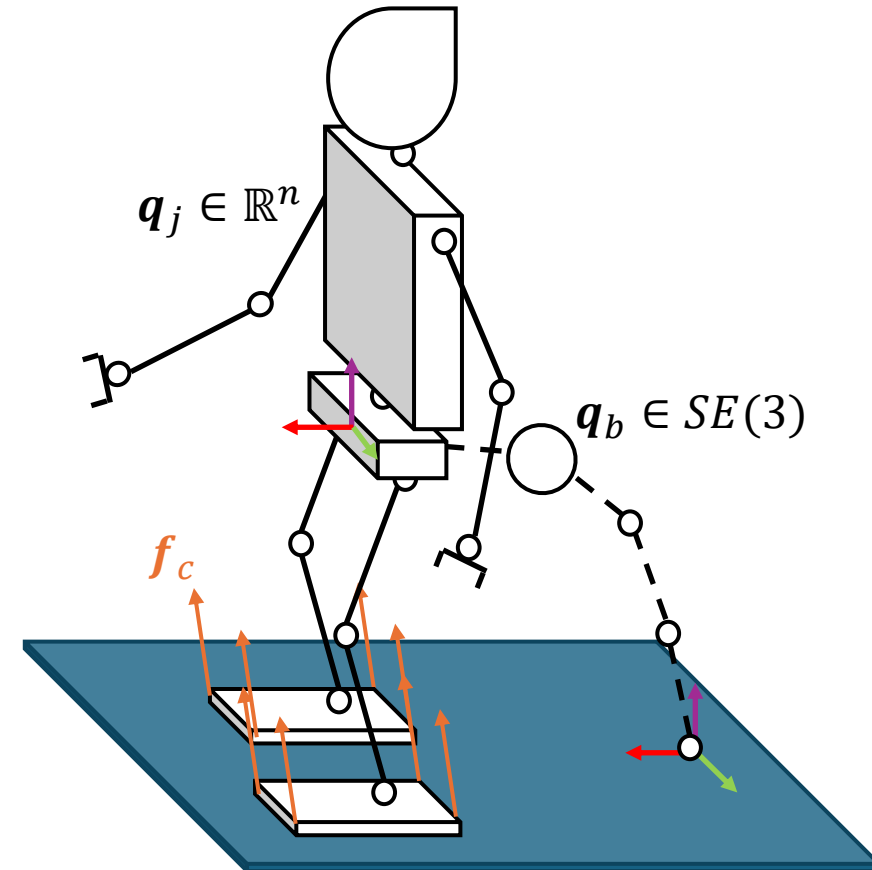
[1] I. Loizou, "Humanoid Bipedal Locomotion through Linear Model Predictive Control", Master Thesis Report, August 2025

 https://github.com/ioloizou/g1_locomotion

Non-Linear Optimal Control and Full-Model

- Template models introduce linearity, but at the cost of reduced accuracy and more complex control structures.
- In contrast, full or quasi-full models offer higher accuracy and simpler control architectures, **but** they introduce significant non-linearities.

$$\begin{cases} \mathbf{M}(\mathbf{q}_k) \dot{\mathbf{v}}_k + \mathbf{h}(\mathbf{q}_k, \mathbf{v}_k) = \mathbf{S} \boldsymbol{\tau} + \sum_{c=0}^{C-1} \mathbf{J}_{c,k}(\mathbf{q}_k)^T \mathbf{f}_{c,k} \\ \mathbf{J}_{c,k}(\mathbf{q}_k) \dot{\mathbf{v}}_k + \dot{\mathbf{J}}_{c,k}(\mathbf{q}_k, \mathbf{v}_k) \mathbf{v}_k = \mathbf{0}, \forall c \in [0, \dots, C) \end{cases}$$



Non-Linear Optimal Control

- Linear (template) models are not accurate and requires complex control architectures with multiple cascade planners/controllers.
- Non-Linear models (e.g. full-model, SRBD, etc..) offer higher accuracy and simpler control architectures.

How we can solve Optimal Control problems involving Non-Linear models (and costs, and constraints)?

$$\begin{aligned} & \min_{\mathbf{w}} f(\mathbf{w}) \\ \text{s. t. } & \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ & \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{aligned}$$

Sequential Quadratic Programming (SQP)

$$\begin{array}{ll} \min_{\mathbf{w}} f(\mathbf{w}) \\ \text{s. t. } \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{array} \longrightarrow \begin{array}{l} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{w}) + \boldsymbol{\lambda}^T \mathbf{e}(\mathbf{w}) + \boldsymbol{\mu}^T \mathbf{n}(\mathbf{w}) \\ \boldsymbol{\mu} \geq \mathbf{0} \end{array}$$

We can compute optimality conditions:

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\mu}} = \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \\ \boldsymbol{\mu} \geq \mathbf{0} \\ \text{diag}(\boldsymbol{\mu}) \mathbf{n}(\mathbf{w}) = \mathbf{0} \end{array} \right.$$

Sequential Quadratic Programming (SQP)

$$\left\{ \begin{array}{l} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\mu}} = \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{array} \right.$$

Let's now consider a linearization of these terms around $\mathbf{w}^*$, $\boldsymbol{\lambda}^*$, and $\boldsymbol{\mu}^*$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

Sequential Quadratic Programming (SQP)

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\mu}} = \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{cases}$$

Let's now consider a linearization of these terms around $\mathbf{w}^*$, $\boldsymbol{\lambda}^*$, and $\boldsymbol{\mu}^*$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

$$\boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \approx \boldsymbol{\psi}^* + \frac{\partial \boldsymbol{\psi}}{\partial \mathbf{w}} \delta \mathbf{w} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\lambda}} \delta \boldsymbol{\lambda} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\mu}} \delta \boldsymbol{\mu}$$

Sequential Quadratic Programming (SQP)

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\mu}} = \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{cases}$$

Let's now consider a linearization of these terms around $\mathbf{w}^*$, $\boldsymbol{\lambda}^*$, and $\boldsymbol{\mu}^*$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

$$\boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \approx \boldsymbol{\psi}^* + \frac{\partial \boldsymbol{\psi}}{\partial \mathbf{w}} \delta \mathbf{w} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\lambda}} \delta \boldsymbol{\lambda} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\mu}} \delta \boldsymbol{\mu} = \boldsymbol{\psi}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} = \mathbf{0}$$

Sequential Quadratic Programming (SQP)

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = \mathbf{e}(\mathbf{w}) = \mathbf{0} \\ \frac{\partial \mathcal{L}}{\partial \boldsymbol{\mu}} = \mathbf{n}(\mathbf{w}) \leq \mathbf{0} \end{cases}$$

Let's now consider a linearization of these terms around $\mathbf{w}^*$, $\boldsymbol{\lambda}^*$, and $\boldsymbol{\mu}^*$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu} = \boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

$$\begin{cases} \boldsymbol{\psi}(\mathbf{w}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \approx \boldsymbol{\psi}^* + \frac{\partial \boldsymbol{\psi}}{\partial \mathbf{w}} \delta \mathbf{w} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\lambda}} \delta \boldsymbol{\lambda} + \frac{\partial \boldsymbol{\psi}}{\partial \boldsymbol{\mu}} \delta \boldsymbol{\mu} = \boldsymbol{\psi}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} = \mathbf{0} \\ \mathbf{e}(\mathbf{w}) \approx \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = \mathbf{0} \\ \mathbf{n}(\mathbf{w}) \approx \mathbf{n}(\mathbf{w}^*) + \frac{\partial \mathbf{n}}{\partial \mathbf{w}} \delta \mathbf{w} \leq \mathbf{0} \end{cases}$$

Sequential Quadratic Programming (SQP)

$$\boldsymbol{\psi}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} = \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda}^* + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} =$$

Sequential Quadratic Programming (SQP)

$$\begin{aligned}\boldsymbol{\psi}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} &= \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda}^* + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu}^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\lambda} + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \boldsymbol{\mu} = \\ &= \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T (\boldsymbol{\lambda}^* + \delta \boldsymbol{\lambda}) + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T (\boldsymbol{\mu}^* + \delta \boldsymbol{\mu}) = \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda}_+ + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \boldsymbol{\mu}_+\end{aligned}$$

Sequential Quadratic Programming (SQP)

$$\begin{aligned}\psi^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \lambda + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \mu &= \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda^* + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \lambda + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \mu = \\ &= \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T (\lambda^* + \delta \lambda) + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T (\mu^* + \delta \mu) = \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda_+ + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu_+\end{aligned}$$

$$\Rightarrow \begin{cases} \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda_+ + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu_+ = \mathbf{0} \\ \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = \mathbf{0} \\ \mathbf{n}(\mathbf{w}^*) + \frac{\partial \mathbf{n}}{\partial \mathbf{w}} \delta \mathbf{w} \leq \mathbf{0} \end{cases}$$

Sequential Quadratic Programming (SQP)

$$\begin{aligned} \psi^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \lambda + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \mu &= \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda^* + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu^* + \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \delta \lambda + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \delta \mu = \\ &= \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T (\lambda^* + \delta \lambda) + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T (\mu^* + \delta \mu) = \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda_+ + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu_+ \end{aligned}$$

$$\begin{aligned} \rightarrow \begin{cases} \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \frac{\partial f}{\partial \mathbf{w}} + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \lambda_+ + \left(\frac{\partial \mathbf{n}}{\partial \mathbf{w}} \right)^T \mu_+ = 0 \\ \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = 0 \\ \mathbf{n}(\mathbf{w}^*) + \frac{\partial \mathbf{n}}{\partial \mathbf{w}} \delta \mathbf{w} \leq 0 \end{cases} & \xrightarrow{\text{Optimality condition of}} \begin{aligned} &\min_{\delta \mathbf{w}} \frac{1}{2} \delta \mathbf{w}^T \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T \delta \mathbf{w} \\ &\text{s.t. } \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = 0 \\ &\quad \mathbf{n}(\mathbf{w}^*) + \frac{\partial \mathbf{n}}{\partial \mathbf{w}} \delta \mathbf{w} \leq 0 \end{aligned} \end{aligned}$$

NOTICE: solving the QP computes an increment $\delta \mathbf{w}$ to estimate the variable $\mathbf{w}$ and the Lagrange multipliers λ_+ and μ_+ associated to $\mathbf{w}$!

SQP Algorithm [1] (for generic NLP)

input: $\mathbf{w}^i = \mathbf{w}^*, \lambda^i = \lambda^*, \mu^i = \mu^*$

$i = 0$

while $i \leq \text{max_iter}$:

 evaluate $\rightarrow f(\mathbf{w}^i), \mathbf{e}(\mathbf{w}^i), \mathbf{n}(\mathbf{w}^i), \frac{\partial f}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2}(\mathbf{w}^i, \lambda^i, \mu^i), \frac{\partial \mathbf{e}}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial \mathbf{n}}{\partial \mathbf{w}}(\mathbf{w}^i)$

 solve QP $\rightarrow \delta \mathbf{w}, \lambda, \mu$

} Construction and resolution of QP
(Possibly exploiting sparsity or structure).

SQP Algorithm [1] (for generic NLP)

input: $\mathbf{w}^i = \mathbf{w}^*, \lambda^i = \lambda^*, \mu^i = \mu^*$

$i = 0$

while $i \leq \text{max_iter}$:

evaluate $\rightarrow f(\mathbf{w}^i), \mathbf{e}(\mathbf{w}^i), \mathbf{n}(\mathbf{w}^i), \frac{\partial f}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2}(\mathbf{w}^i, \lambda^i, \mu^i), \frac{\partial \mathbf{e}}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial \mathbf{n}}{\partial \mathbf{w}}(\mathbf{w}^i)$

solve QP $\rightarrow \delta \mathbf{w}, \lambda, \mu$

Construction and resolution of QP
(Possibly exploiting sparsity or structure).

$\alpha = 1$

while $\alpha \leq \alpha_{\min}$:

$\mathbf{w} = \mathbf{w}^i + \alpha \delta \mathbf{w}$

$\text{candidate_merit} = \text{compute_candidate_merit}(\mathbf{w}, \lambda, \mu)$

$\text{merit} = \text{compute_merit}(\mathbf{w}^i, \lambda^i, \mu^i)$

if $\text{candidate_merit} \leq \text{merit}$:

$$\begin{bmatrix} \mathbf{w}^i \\ \lambda^i \\ \mu^i \end{bmatrix} = \begin{bmatrix} \mathbf{w} \\ \lambda \\ \mu \end{bmatrix}$$

break

else:

$$\alpha = \frac{\alpha}{2}$$

Line-search:

determine the optimal value
of α for the Newton step to
ensure a decreasing in cost
while keeping constraints.

SQP Algorithm [1] (for generic NLP)

input: $\mathbf{w}^i = \mathbf{w}^*, \lambda^i = \lambda^*, \mu^i = \mu^*$

$i = 0$

while $i \leq \text{max_iter}$:

evaluate $\rightarrow f(\mathbf{w}^i), \mathbf{e}(\mathbf{w}^i), \mathbf{n}(\mathbf{w}^i), \frac{\partial f}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2}(\mathbf{w}^i, \lambda^i, \mu^i), \frac{\partial \mathbf{e}}{\partial \mathbf{w}}(\mathbf{w}^i), \frac{\partial \mathbf{n}}{\partial \mathbf{w}}(\mathbf{w}^i)$
solve QP $\rightarrow \delta \mathbf{w}, \lambda, \mu$

Construction and resolution of QP
(Possibly exploiting sparsity or structure).

$\alpha = 1$

while $\alpha \leq \alpha_{\min}$:

$\mathbf{w} = \mathbf{w}^i + \alpha \delta \mathbf{w}$

$\text{candidate_merit} = \text{compute_candidate_merit}(\mathbf{w}, \lambda, \mu)$

$\text{merit} = \text{compute_merit}(\mathbf{w}^i, \lambda^i, \mu^i)$

if $\text{candidate_merit} \leq \text{merit}$:

$\begin{bmatrix} \mathbf{w}^i \\ \lambda^i \\ \mu^i \end{bmatrix} = \begin{bmatrix} \mathbf{w} \\ \lambda \\ \mu \end{bmatrix}$

break

else:

$\alpha = \frac{\alpha}{2}$

Line-search:

determine the optimal value
of α for the Newton step to
ensure a decreasing in cost
while keeping constraints.

if $\text{is_optimal}(\mathbf{w}^i, \lambda^i, \mu^i)$:

break

Evaluates optimality on original NLP

SQP Gauss-Newton Hessian Approximation

$$\begin{array}{ll} \min_{\mathbf{w}} & \frac{1}{2} \|f(\mathbf{w})\|_2^2 \\ \text{s. t.} & \mathbf{e}(\mathbf{w}) = \mathbf{0} \end{array}$$



$$\begin{aligned} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) &= \frac{1}{2} f(\mathbf{w})^T f(\mathbf{w}) + \boldsymbol{\lambda}^T \mathbf{e}(\mathbf{w}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T f(\mathbf{w}) + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} \end{aligned}$$

SQP Gauss-Newton Hessian Approximation

$$\begin{array}{ll} \min_{\mathbf{w}} & \frac{1}{2} \|f(\mathbf{w})\|_2^2 \\ \text{s. t.} & \mathbf{e}(\mathbf{w}) = \mathbf{0} \end{array} \quad \Rightarrow \quad \begin{array}{l} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) = \frac{1}{2} f(\mathbf{w})^T f(\mathbf{w}) + \boldsymbol{\lambda}^T \mathbf{e}(\mathbf{w}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T f(\mathbf{w}) + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} \end{array}$$

- The QP for the linear approximation will be:

$$\begin{array}{ll} \min_{\delta \mathbf{w}} & \frac{1}{2} \delta \mathbf{w}^T \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T \delta \mathbf{w} \\ \text{s. t.} & \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = \mathbf{0} \end{array} \quad \Rightarrow \quad \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} = \underbrace{\left(\frac{\partial f}{\partial \mathbf{w}} \right)^T \frac{\partial f}{\partial \mathbf{w}}}_{1^{\text{st}} \text{ order term}} + \underbrace{\left(\frac{\partial^2 f}{\partial \mathbf{w}^2} \right)^T f(\mathbf{w}) + \left(\frac{\partial^2 \mathbf{e}}{\partial \mathbf{w}^2} \right)^T \boldsymbol{\lambda}}_{2^{\text{nd}} \text{ order terms}}$$

SQP Gauss-Newton Hessian Approximation

$$\begin{aligned} \min_{\mathbf{w}} \quad & \frac{1}{2} \|f(\mathbf{w})\|_2^2 \\ \text{s.t.} \quad & \mathbf{e}(\mathbf{w}) = \mathbf{0} \end{aligned} \quad \Rightarrow \quad \begin{aligned} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) &= \frac{1}{2} f(\mathbf{w})^T f(\mathbf{w}) + \boldsymbol{\lambda}^T \mathbf{e}(\mathbf{w}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T f(\mathbf{w}) + \left(\frac{\partial \mathbf{e}}{\partial \mathbf{w}} \right)^T \boldsymbol{\lambda} \end{aligned}$$

- The QP for the linear approximation will be:

$$\begin{aligned} \min_{\delta \mathbf{w}} \quad & \frac{1}{2} \delta \mathbf{w}^T \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} \delta \mathbf{w} + \left(\frac{\partial f}{\partial \mathbf{w}} \right)^T \delta \mathbf{w} \\ \text{s.t.} \quad & \mathbf{e}(\mathbf{w}^*) + \frac{\partial \mathbf{e}}{\partial \mathbf{w}} \delta \mathbf{w} = \mathbf{0} \end{aligned} \quad \Rightarrow \quad \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w}^2} = \underbrace{\left(\frac{\partial f}{\partial \mathbf{w}} \right)^T \frac{\partial f}{\partial \mathbf{w}}}_{1^{\text{st}} \text{ order term}} + \underbrace{\left(\frac{\partial^2 f}{\partial \mathbf{w}^2} \right)^T f(\mathbf{w}) + \left(\frac{\partial^2 \mathbf{e}}{\partial \mathbf{w}^2} \right)^T \boldsymbol{\lambda}}_{2^{\text{nd}} \text{ order terms}}$$

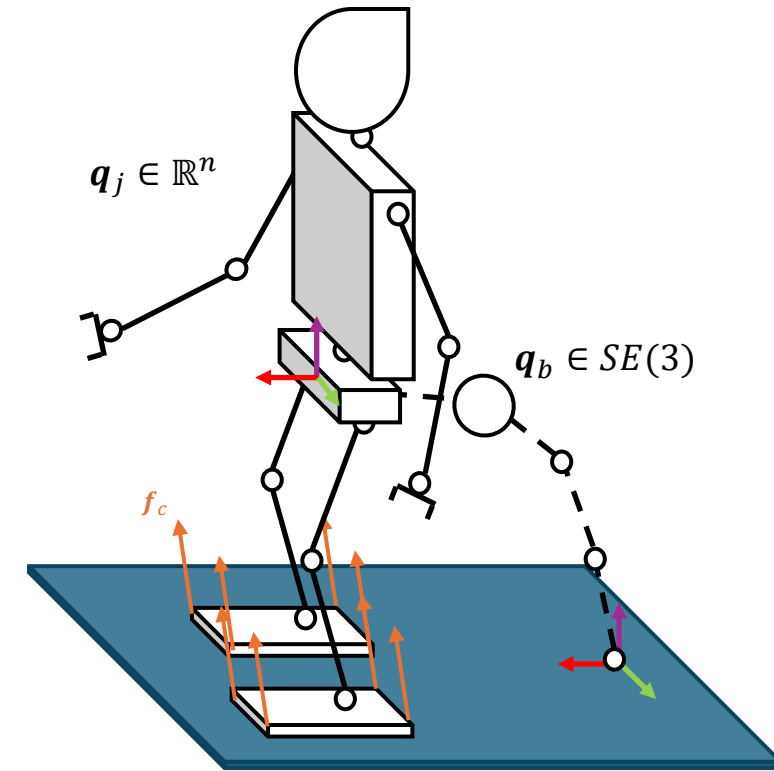
Gauss-Newton method approximates Hessian **NOT** considering 2nd order terms in the exact Hessian.

SQP and Full-Model (Forward Dynamics)

$$\left\{ \begin{array}{l} x = \left\{ \begin{array}{l} \mathbf{p}_b \\ \boldsymbol{\rho}_b \\ \mathbf{q}_j \\ \dot{\mathbf{p}}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_j \end{array} \right\} \in \mathbb{R}^{2n+6} \times \mathbb{R}^3 \times SO(3), \quad \dot{x} = \left\{ \begin{array}{l} \dot{\mathbf{p}}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_j \\ \ddot{\mathbf{p}}_b \\ \dot{\boldsymbol{\omega}}_b \\ \ddot{\mathbf{q}}_j \end{array} \right\} \in \mathbb{R}^{2n+12}, \quad \mathbf{u} = \left[\begin{array}{c} \boldsymbol{\tau} \\ \mathbf{f}_0 \\ \vdots \\ \mathbf{f}_{c-1} \end{array} \right] \in \mathbb{R}^{n+3c} \\ \|\boldsymbol{\rho}_b\| = 1, \end{array} \right. \quad \dot{\mathbf{p}}_b = \boldsymbol{\rho}_b \circ \left[\begin{array}{c} 1 \\ \frac{1}{2}\boldsymbol{\omega}_b \\ 0 \end{array} \right]$$

$$\Rightarrow \dot{\mathbf{v}} = \mathbf{M}(\mathbf{q})^{-1} \left(\mathbf{S}\boldsymbol{\tau} + \sum_{c=0}^{c-1} \mathbf{J}_c(\mathbf{q})^T \mathbf{f}_c - \mathbf{h}(\mathbf{q}, \mathbf{v}) \right)$$

Enters directly in the integrator.



SQP and Full-Model (Inverse Dynamics)

$$\left\{ \begin{array}{l} x = \left\{ \begin{array}{l} \mathbf{p}_b \\ \boldsymbol{\rho}_b \\ \mathbf{q}_j \\ \dot{\mathbf{p}}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_j \end{array} \right\} \in \mathbb{R}^{2n+6} \times \mathbb{R}^3 \times SO(3) \\ \left\{ \begin{array}{l} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\rho}}_b \end{array} \right\} = \mathbf{v} \\ \|\boldsymbol{\rho}_b\| = 1, \end{array} \right. \quad \dot{x} = \left\{ \begin{array}{l} \dot{\mathbf{p}}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_j \\ \ddot{\mathbf{p}}_b \\ \dot{\boldsymbol{\omega}}_b \\ \ddot{\mathbf{q}}_j \end{array} \right\} \in \mathbb{R}^{2n+12}, \quad \mathbf{u} = \left\{ \begin{array}{l} \ddot{\mathbf{p}}_b \\ \dot{\boldsymbol{\omega}}_b \\ \ddot{\mathbf{q}}_j \\ \mathbf{f}_0 \\ \vdots \\ \mathbf{f}_{C-1} \end{array} \right\} \in \mathbb{R}^{n+6+3C}$$

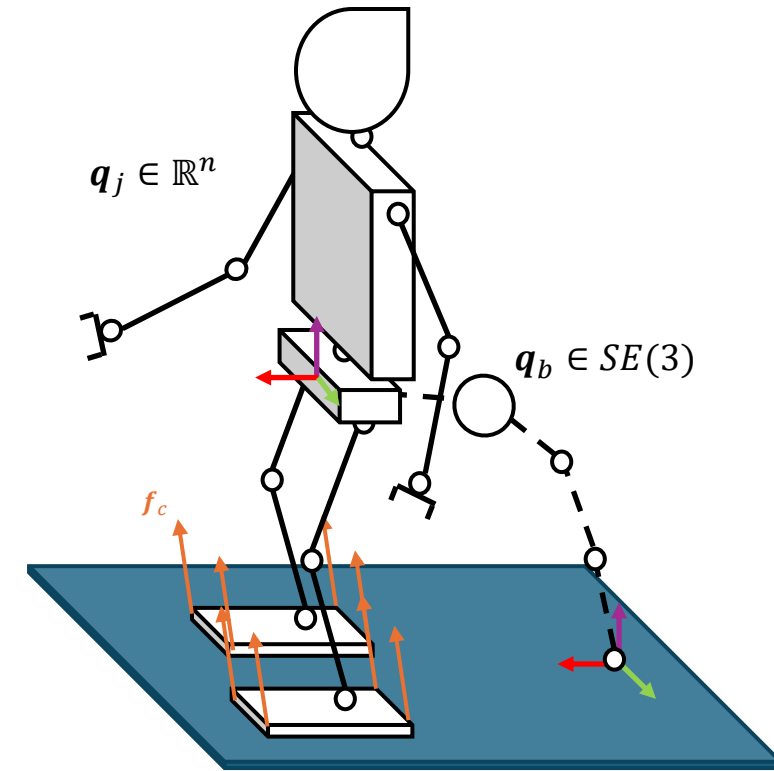
$$\dot{\boldsymbol{\rho}}_b = \boldsymbol{\rho}_b \circ \begin{bmatrix} 1 \\ \frac{1}{2}\boldsymbol{\omega}_b \\ 0 \end{bmatrix}$$

$$\Rightarrow \mathbf{M}_b(\mathbf{q})\dot{\mathbf{v}} + \mathbf{h}_b(\mathbf{q}, \mathbf{v}) = \sum_{c=0}^{C-1} \mathbf{J}_{b,c}(\mathbf{q})^T \mathbf{f}_c$$

Became an equality constraint.

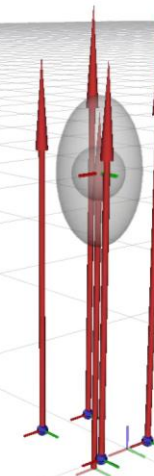
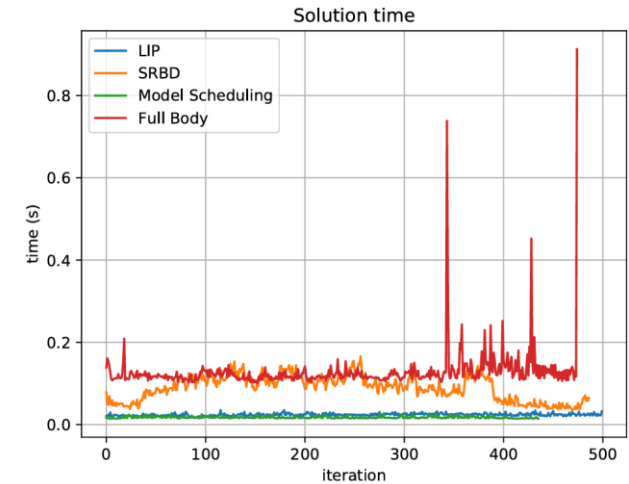
$$\mathbf{J}_c(\mathbf{q}) = \begin{bmatrix} \mathbf{J}_{b,c}(\mathbf{q}) & \mathbf{J}_{j,c}(\mathbf{q}) \end{bmatrix}$$

$\in \mathbb{R}^{6 \times 6} \quad \in \mathbb{R}^{6 \times n}$



SQP and Full-Model

- Derivatives of kinematics and dynamics can be computed using automatic differentiation (see CasADi [1]) or using dedicated libraries (such as Pinocchio [2]).
- Other formulations:
 - Kino-Dynamic (Centroidal Dynamics + IK) [3]
 - Compact-ID (Express base accelerations in terms of joint accelerations → avoid equality constraint) [4]
 - ...
- *Model-Scheduling* can be used to speed-up the optimization [5]
- Optimize over total time $T = \sum dt_k$ or dt_k



- [1] Andersson, Joel AE, et al. "CasADi: a software framework for nonlinear optimization and optimal control." *Mathematical Programming Computation* 11 (2019): 1-36.
- [2] Carpentier, Justin, et al. "The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives." *2019 IEEE/SICE International Symposium on System Integration (SII)*. IEEE, 2019.
- [3] Kuindersma, S., Deits, R., Fallon, M. et al. Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot. *Auton Robot* **40**, 429–455 (2016)
- [4] S. Katayama and T. Ohtsuka, "Efficient solution method based on inverse dynamics for optimal control problems of rigid body systems," 2021 IEEE International Conference on Robotics and Automation (ICRA), 2021, pp. 2070–2076
- [5] H Li, PM Wensing, "CAFE-MPC: A cascaded-fidelity model predictive control framework with tuning-free whole-body control", IEEE Transactions on Robotics 41, 837 - 856

Bits On Orientation Coordinates [1]

Euler Angles

- $\mathbf{q} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\theta}_b \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^{6+n}$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\theta}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{6+n}$

➡ $\mathbf{q}_{k+1} = \mathbf{q}_k + dt \dot{\mathbf{q}}_k$

- $\mathbf{q}_k^{i+1} = \mathbf{q}_k^i + \delta \mathbf{q}_k^{i+1}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \begin{bmatrix} \delta \mathbf{p}_b \\ \delta \boldsymbol{\theta}_b \\ \delta \mathbf{q}_j \end{bmatrix},$$

Derivatives can be computed using AD (e.g. casadi). Suffers singularities in the representation.

Bits On Orientation Coordinates [1]

Euler Angles

- $\mathbf{q} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\theta}_b \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^{6+n}$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\theta}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{6+n}$

$$\Rightarrow \mathbf{q}_{k+1} = \mathbf{q}_k + dt \dot{\mathbf{q}}_k$$

- $\mathbf{q}_k^{i+1} = \mathbf{q}_k^i + \delta \mathbf{q}_k^{i+1}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \begin{bmatrix} \delta \mathbf{p}_b \\ \delta \boldsymbol{\theta}_b \\ \delta \mathbf{q}_j \end{bmatrix},$$

Derivatives can be computed using AD (e.g. casadi). Suffers singularities in the representation.

Quaternions

- $\mathbf{q} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\rho}_b \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^3 \times \{\mathbb{R}^4: \|\boldsymbol{\rho}_b\| = 1\} \times \mathbb{R}^n$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\rho}}_b = \boldsymbol{\rho}_{b,k} \circ \begin{bmatrix} 0.5 \boldsymbol{\omega}_{b,k} \\ 0 \end{bmatrix} \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{7+n}$

$$\Rightarrow \mathbf{q}_{k+1} = \begin{bmatrix} \mathbf{p}_{b,k} + dt \dot{\mathbf{p}}_{b,k} \\ \boldsymbol{\rho}_{b,k} + \boldsymbol{\rho}_{b,k} \circ \begin{bmatrix} 0.5 \boldsymbol{\omega}_{b,k} \\ 0 \end{bmatrix} dt \\ \mathbf{q}_{j,k} + dt \dot{\mathbf{q}}_{j,k} \end{bmatrix}$$

- $\mathbf{q}_k^{i+1} = \mathbf{q}_k^i + \delta \mathbf{q}_k^{i+1}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \begin{bmatrix} \delta \mathbf{p}_b \\ \delta \boldsymbol{\rho}_b \\ \delta \mathbf{q}_j \end{bmatrix}$$

Derivatives can be computed using AD (e.g. casadi). $\boldsymbol{\rho}_b$ should be constrained $\|\boldsymbol{\rho}_b\| = 1$.

Bits On Orientation Coordinates [1]

Euler Angles

- $\mathbf{q} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\theta}_b \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^{6+n}$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\theta}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{6+n}$

➡ $\mathbf{q}_{k+1} = \mathbf{q}_k + dt \dot{\mathbf{q}}_k$

- $\mathbf{q}_k^{i+1} = \mathbf{q}_k^i + \delta \mathbf{q}_k^{i+1}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \begin{bmatrix} \delta \mathbf{p}_b \\ \delta \boldsymbol{\theta}_b \\ \delta \mathbf{q}_j \end{bmatrix},$$

Derivatives can be computed using AD (e.g. casadi). Suffers singularities in the representation.

Quaternions

- $\mathbf{q} = \begin{bmatrix} \mathbf{p}_b \\ \boldsymbol{\rho}_b \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^3 \times \{\mathbb{R}^4: \|\boldsymbol{\rho}_b\| = 1\} \times \mathbb{R}^n$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\mathbf{p}}_b \\ \dot{\boldsymbol{\rho}}_b = \boldsymbol{\rho}_{b,k} \circ \begin{bmatrix} 0.5 \boldsymbol{\omega}_{b,k} \\ 0 \end{bmatrix} \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{7+n}$

➡ $\mathbf{q}_{k+1} = \begin{bmatrix} \mathbf{p}_{b,k} + dt \dot{\mathbf{p}}_{b,k} \\ \boldsymbol{\rho}_{b,k} + \boldsymbol{\rho}_{b,k} \circ \begin{bmatrix} 0.5 \boldsymbol{\omega}_{b,k} \\ 0 \end{bmatrix} dt \\ \mathbf{q}_{j,k} + dt \dot{\mathbf{q}}_{j,k} \end{bmatrix}$

- $\mathbf{q}_k^{i+1} = \mathbf{q}_k^i + \delta \mathbf{q}_k^{i+1}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + \frac{\partial f(\mathbf{q})}{\partial \mathbf{q}} \begin{bmatrix} \delta \mathbf{p}_b \\ \delta \boldsymbol{\rho}_b \\ \delta \mathbf{q}_j \end{bmatrix}$$

Derivatives can be computed using AD (e.g. casadi). $\boldsymbol{\rho}_b$ should be constrained $\|\boldsymbol{\rho}_b\| = 1$.

SE(3)

- $\mathbf{q} = (\mathbf{T}_b, \mathbf{q}_j) \in SE(3) \times \mathbb{R}^n$
- $\dot{\mathbf{q}} = \begin{bmatrix} \dot{\boldsymbol{\xi}}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathfrak{se}(3) \times \mathbb{R}^n$
- ➡ $\mathbf{q}_{k+1} = \begin{bmatrix} \mathbf{T}_{b,k} \exp(dt \hat{\boldsymbol{\xi}}_b) \\ \mathbf{q}_{j,k} + dt \dot{\mathbf{q}}_{j,k} \end{bmatrix}$
- $\mathbf{q}_k^i = \begin{bmatrix} \mathbf{T}_{b,k}^i \exp(\delta \hat{\boldsymbol{\xi}}_{b,k}^i) \\ \mathbf{q}_{j,k}^i + \delta \mathbf{q}_{j,k}^i \end{bmatrix}$
- $f(\mathbf{q}) \in \mathbb{R}^m$

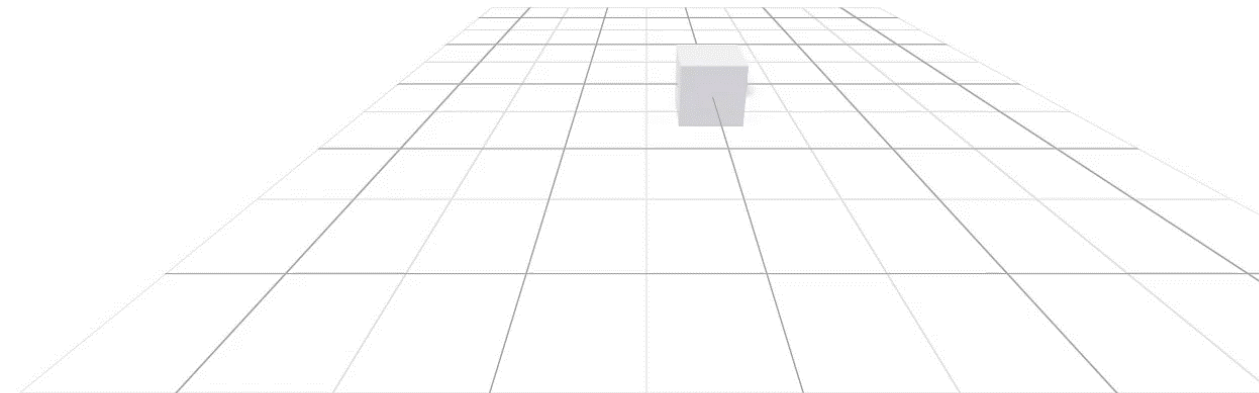
$$f(\mathbf{q}) \approx f(\mathbf{q}^*) + J(\mathbf{q}) \begin{bmatrix} \delta \boldsymbol{\xi}_b \\ \delta \mathbf{q}_j \end{bmatrix}$$

Derivatives $J(\mathbf{q})$ can be computed using Pinocchio. Needs to use sum in the manifold for Newton step.

TrajOpt in SE(3)

$N_s = 100$

$$e(T_b) \in \mathbb{R}^3 \times \mathfrak{so}(3)$$

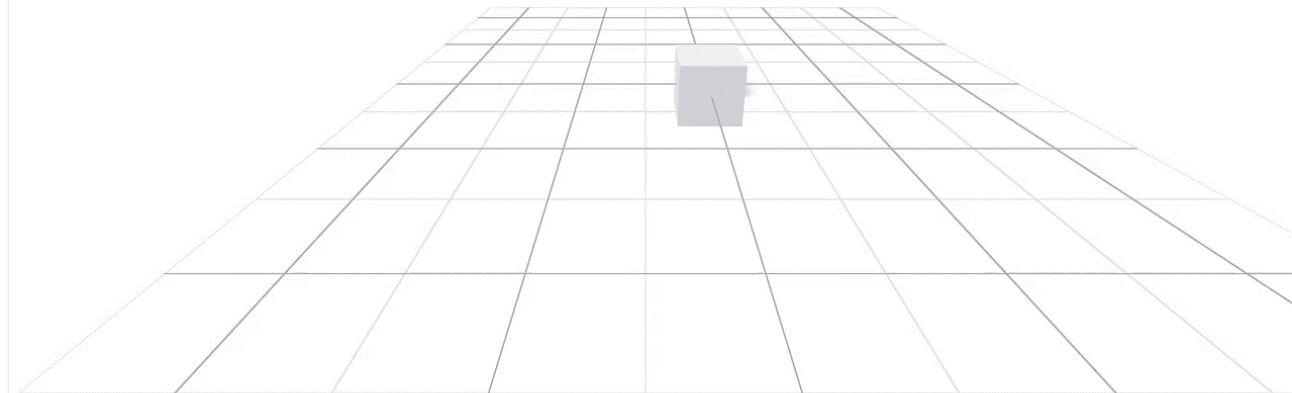


Iter	Cost	Violation	Defect	DualInf	d	LS	QP[s/i]	t(ms)
1	2.81e+03	0.00e+00	7.30e-09	3.14e+03	3.14e+00	1	0 / 2	1.485
2	6.83e-23	0.00e+00	2.92e-01	2.00e+03	2.10e+00	1	0 / 1	0.774
3	4.60e-25	0.00e+00	3.33e-03	3.56e-10	3.09e-01	1	0 / 2	1.203

=== SQP Solver Statistics ===

Total Cost: 8.74e-29
Constraint Violation: 0.00e+00
Dynamics Defect: 3.36e-07
Dual Infeasibility: 2.24e-11
Linesearch Iterations: 1
QP info: OK in 1 iters
Last Iteration Time: 0.744 ms
Total Time: 0.004 s

$$e(T_b) \in \mathfrak{se}(3)$$

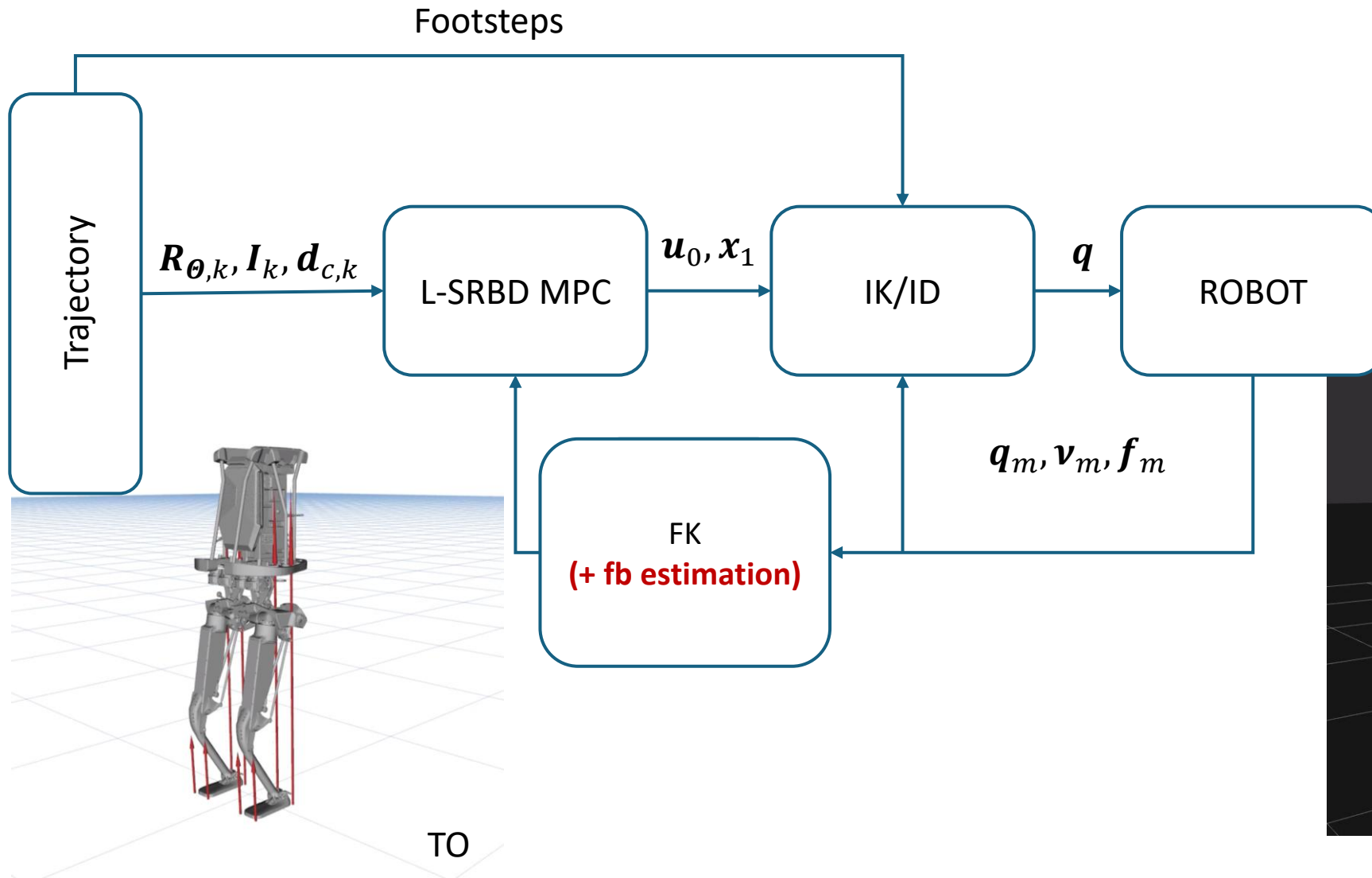


Iter	Cost	Violation	Defect	DualInf	d	LS	QP[s/i]	t(ms)
1	1.47e-22	0.00e+00	2.51e-04	6.28e+03	3.14e+00	1	0 / 1	1.054

=== SQP Solver Statistics ===

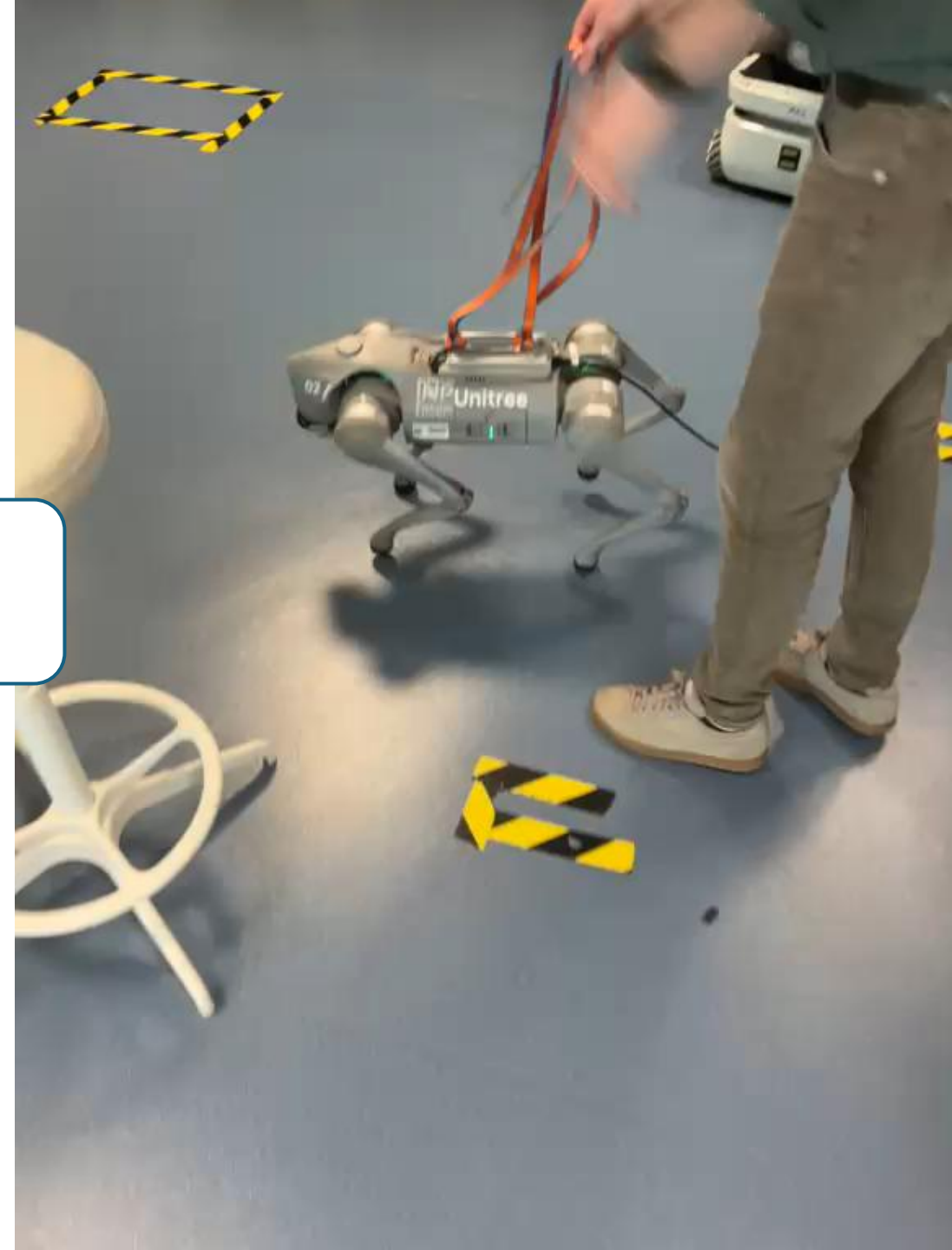
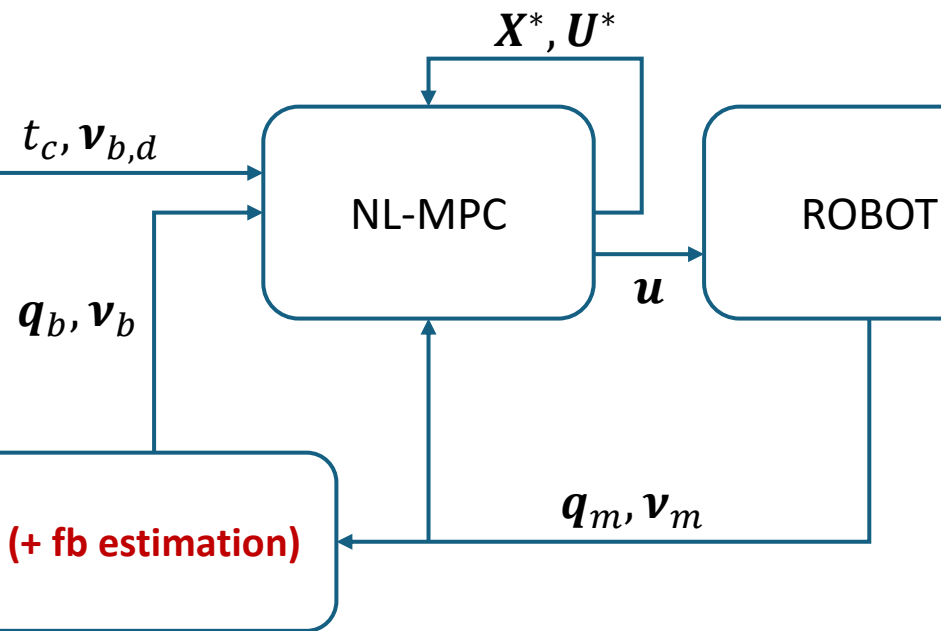
Total Cost: 5.02e-31
Constraint Violation: 0.00e+00
Dynamics Defect: 3.57e-09
Dual Infeasibility: 4.64e-10
Linesearch Iterations: 1
QP info: OK in 1 iters
Last Iteration Time: 0.770 ms
Total Time: 0.002 s

Non-Linear TO & L-SRBD MPC



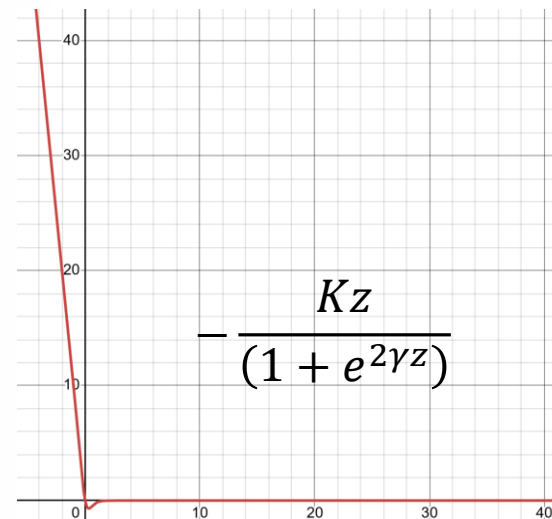
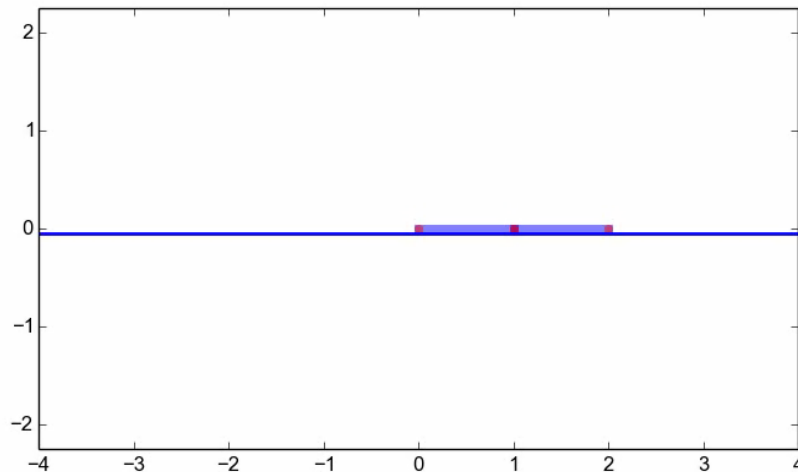
NL-MPC through SQP

- Base Velocity Reference
- Gait timings

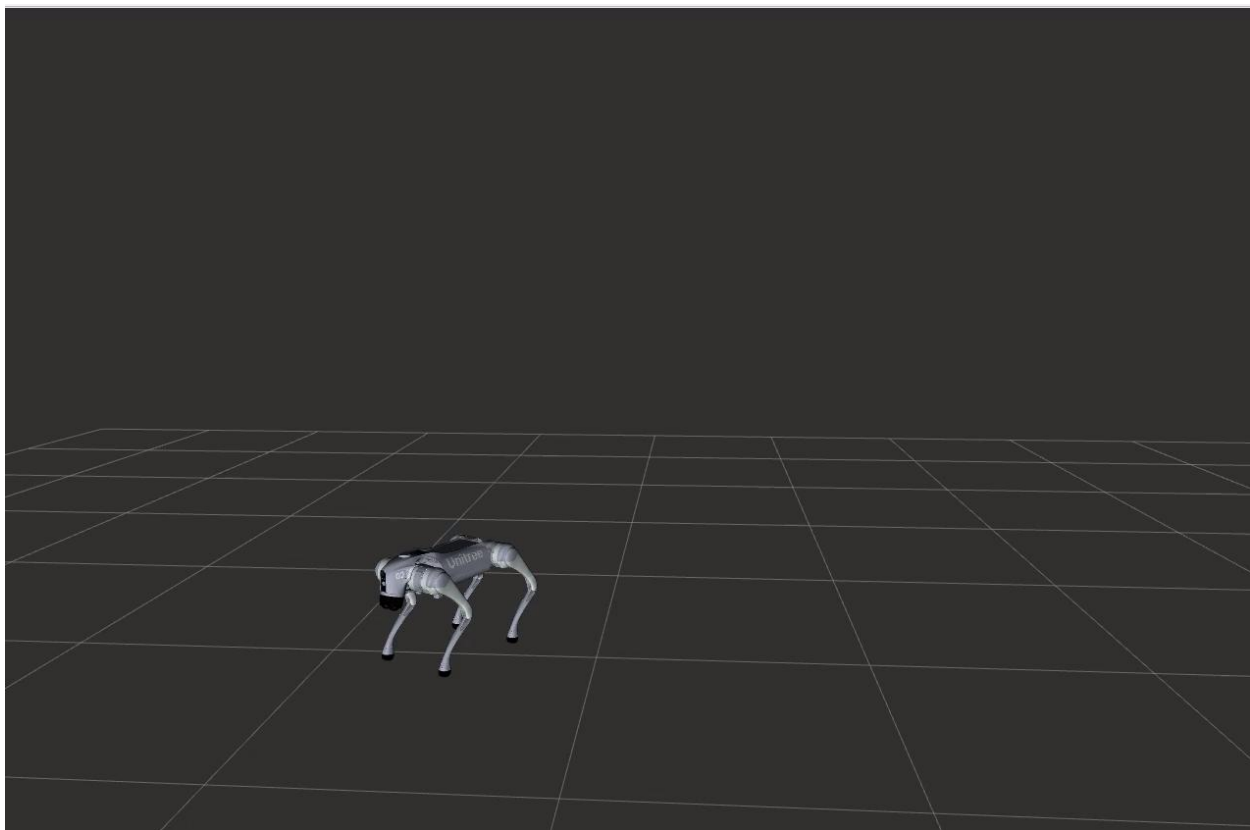


SQP MPC

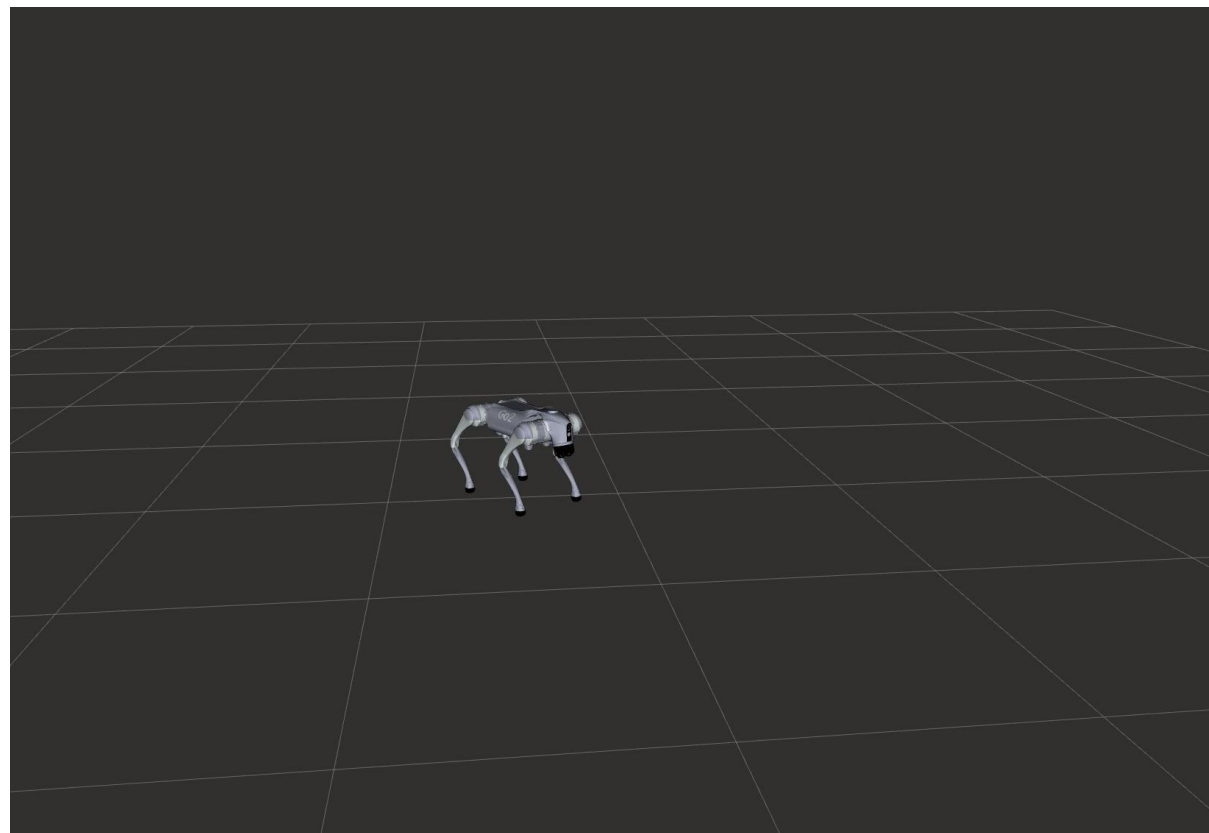
- It is not needed to resolve the optimization problem for many iterations **if a good guess is provided**. We can linearize on the previous optimized trajectory, hence $max_iter = 1$.
- Contact location is also optimized; contact-z follows simple heuristic.
- **Contact schedule is given:**
 - Contact Explicit requires contact schedule planner
 - Contact Implicit automatically discover contact phases **BUT** requires special solvers capable to handle **complementarity constraints**.



SQP MPC & TO Examples in SE(3)

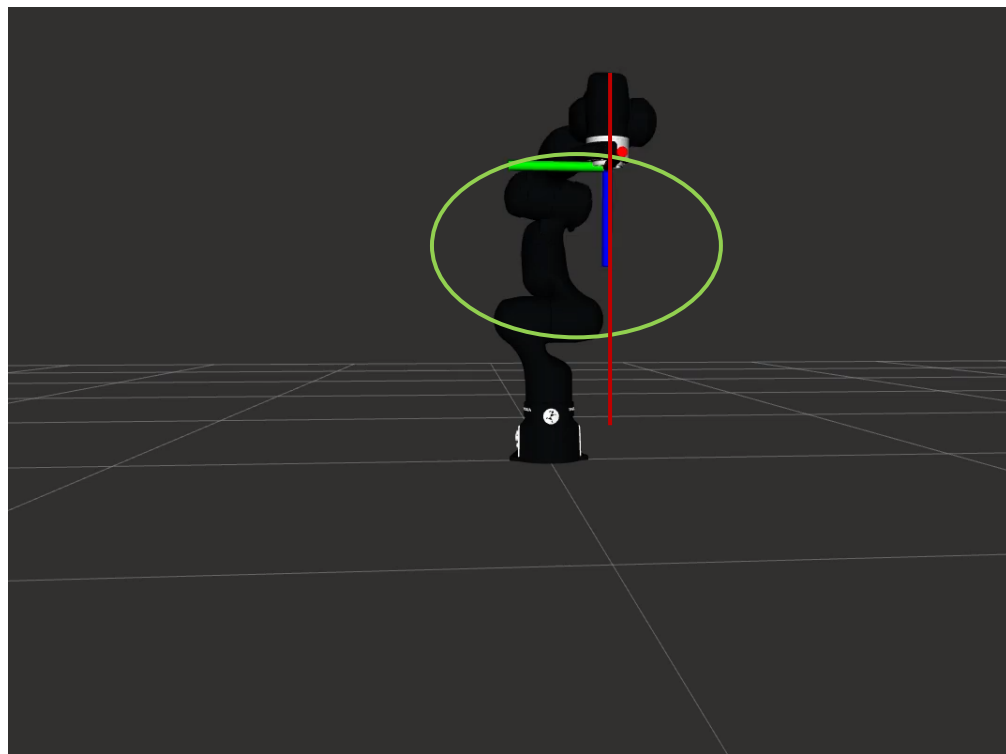


Side flip (TO)

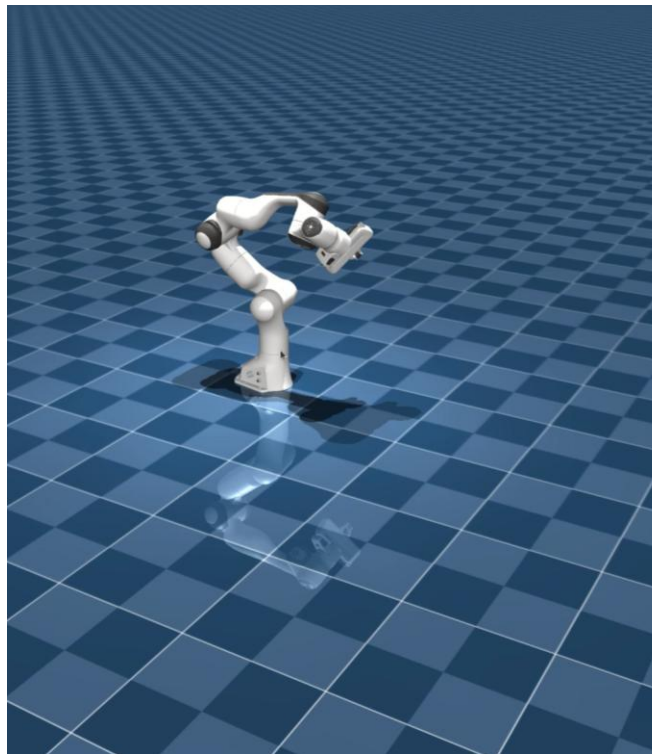


Locomotion (MPC)

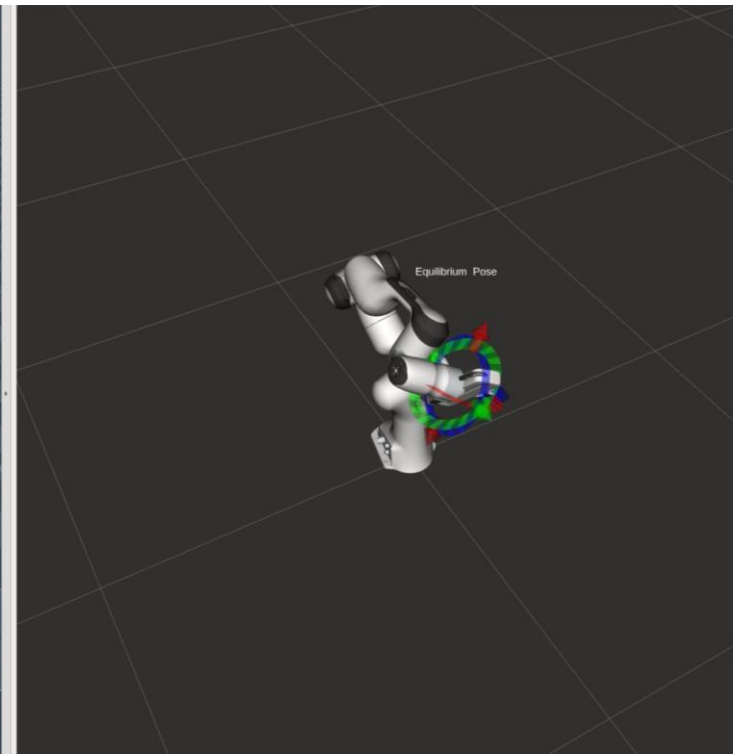
SQP MPC Examples



MPC + Cartesian Constraint (barrier)

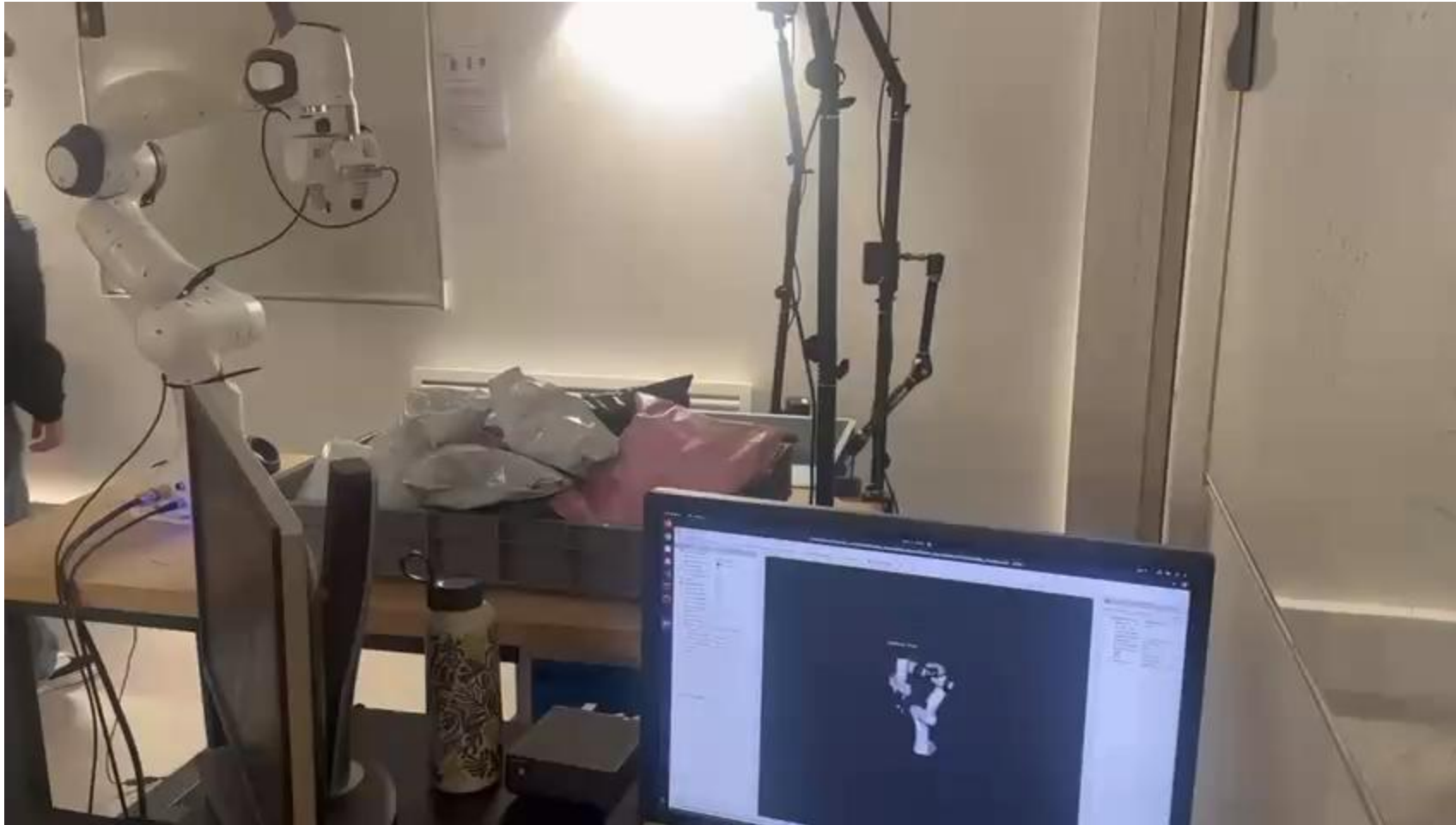


Mujoco Simulator

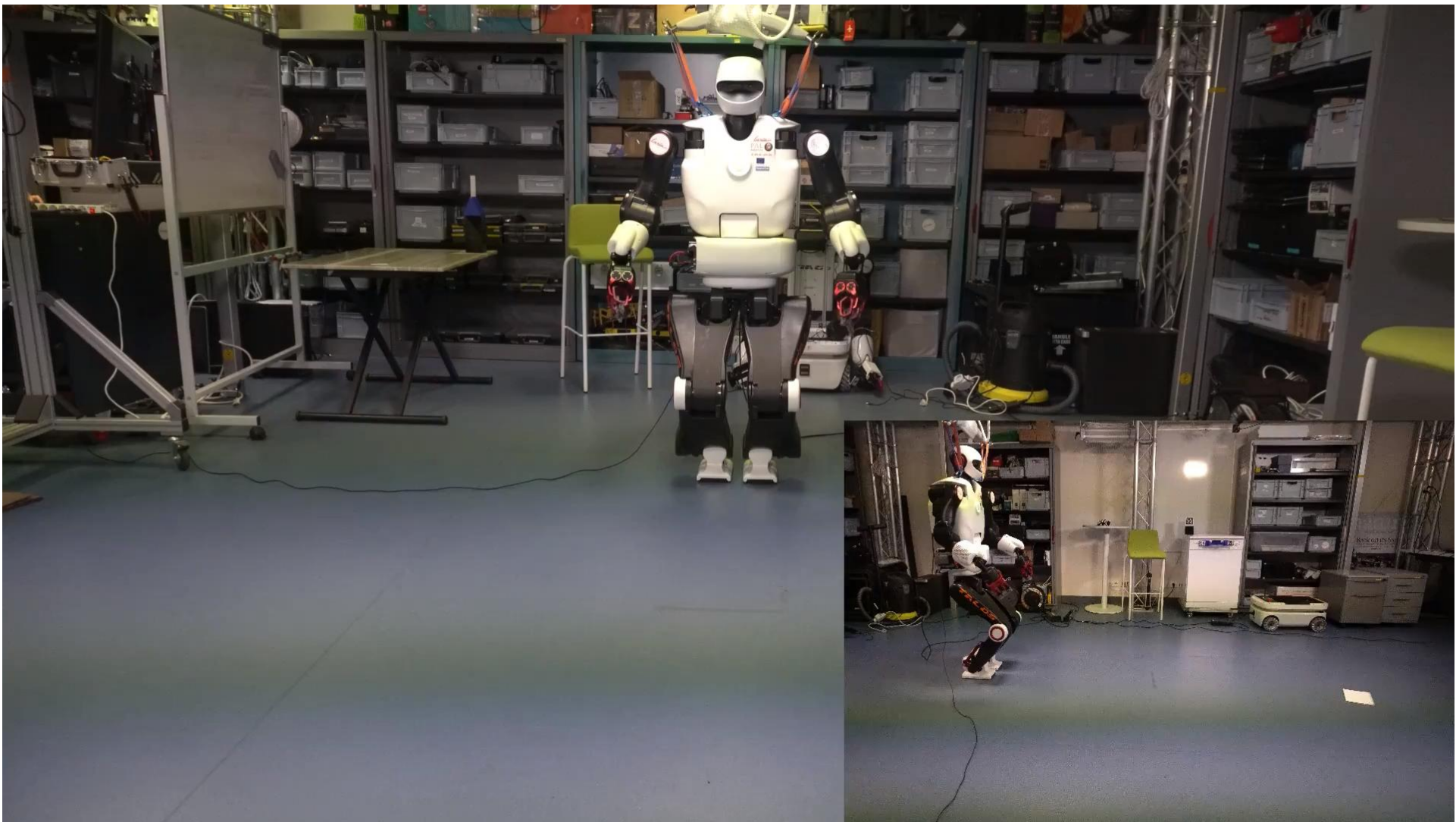


MPC + Interactive Marker

SQP MPC Examples



Online Full-Body iLQR



MPC and TO: Software

- **Horizon:** <https://github.com/ADVRHumanoids/horizon>
- **Croccodyl:** <https://github.com/loco-3d/croccodyl>
- **Aligator:** <https://github.com/Simple-Robotics/aligator>
- **CasADi/acados:** <https://github.com/casadi/casadi> / <https://github.com/acados/acados>
- **OCS2:** <https://github.com/leggedrobotics/ocs2>
- **se3_sqp:** https://github.com/hucebot/se3_sqp

Bi-Level Optimization for Contact Implicit TO

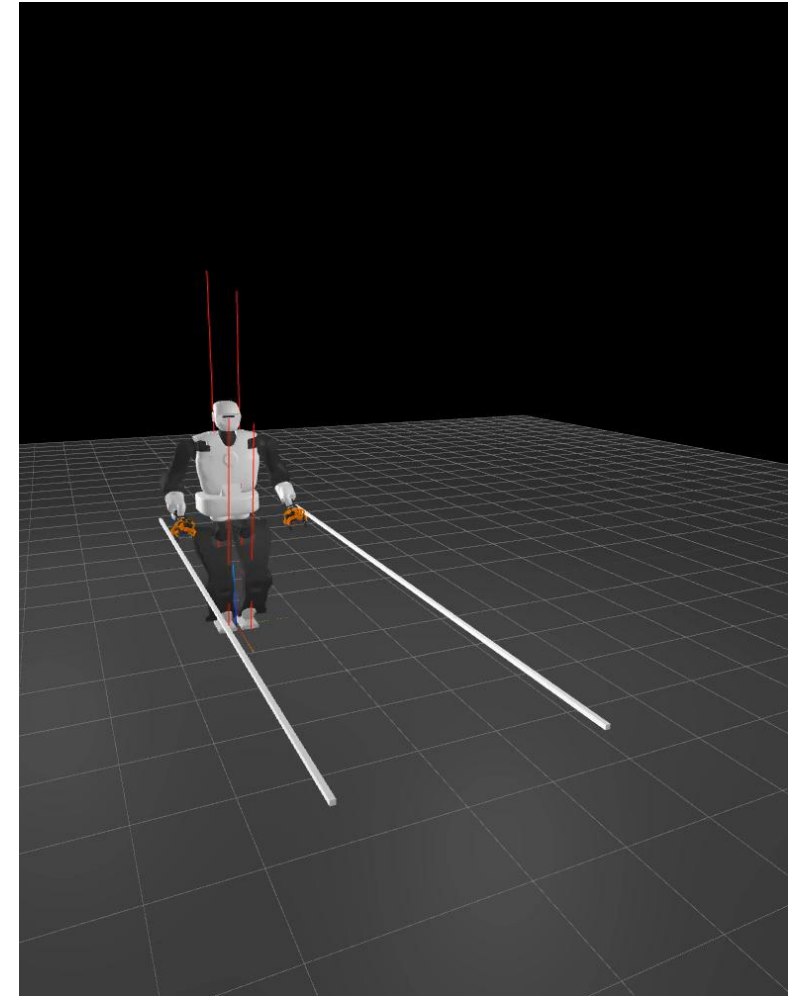
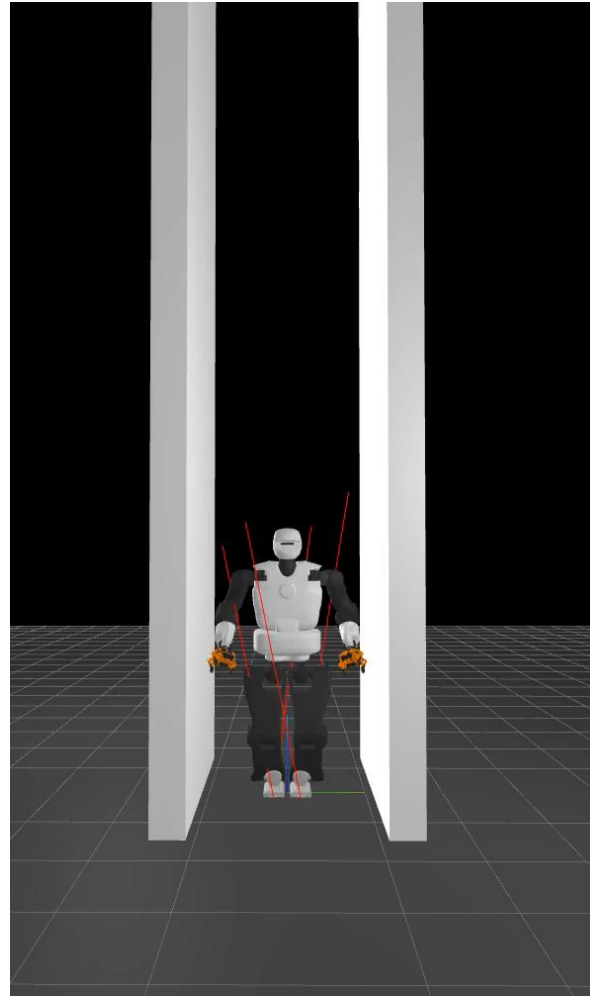
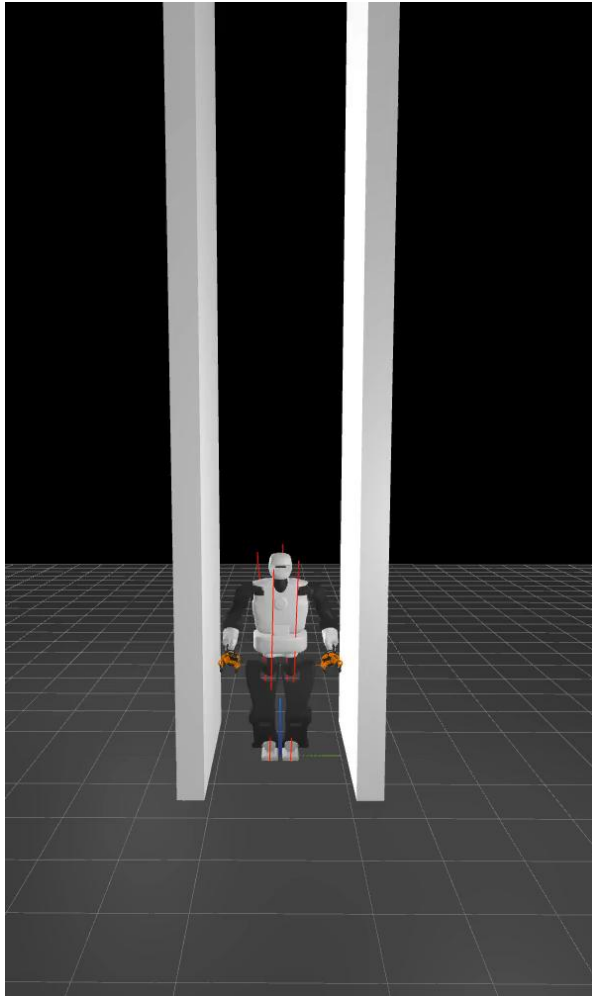
$$\begin{aligned} \min_{\mathbf{Z}, \mathbf{W}} & \{f(\mathbf{Z}_1, \mathbf{W}_1), f(\mathbf{Z}_2, \mathbf{W}_2), \dots, f(\mathbf{Z}_E, \mathbf{W}_E)\} \\ \text{s. t.} & \quad \mathbf{Z}_j \in \mathbb{Z}^{n_z}, \forall j = \{1, 2, \dots, E\} \\ & \quad \mathbf{W}_j \in \mathbb{R}^{n_w}, \forall j = \{1, 2, \dots, E\} \\ & \quad \mathbf{Z}_m \leq \mathbf{Z}_j \leq \mathbf{Z}_M, \forall j = \{1, 2, \dots, E\} \\ & \quad f(\mathbf{Z}_j, \mathbf{W}_j) = h(\mathbf{Z}_j) + \left\{ \min_{\mathbf{W}} l(\mathbf{Z}_j, \mathbf{W}_j) \right\}, \forall j = \{1, 2, \dots, E\} \\ & \quad \text{s. t.} \quad \dots \end{aligned}$$

- **Outer-Loop** optimizer handles integer variables $\mathbf{Z}_j$ and evaluates multiple *instances* j of the inner-loop optimization. 0-order optimizers can be used, i.e. ACO [1] or CMAES.
- **Inner-Loop** optimizer solves the continuous problem in the $\mathbf{W}_j$ variables, parameterized in $\mathbf{Z}_j$.

[1] T. Liao, K. Socha, M. A. Montes de Oca, T. Stützle, and M. Dorigo, “Ant Colony Optimization for Mixed-Variable Optimization Problems,” *IEEE Transactions on Evolutionary Computation*, 18(4), 503–518, 2014.

[2] N. Hansen and A. Ostermeier, “Completely Derandomized Self-Adaptation in Evolution Strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

Bi-Level Optimization for Contact Implicit TO [1]

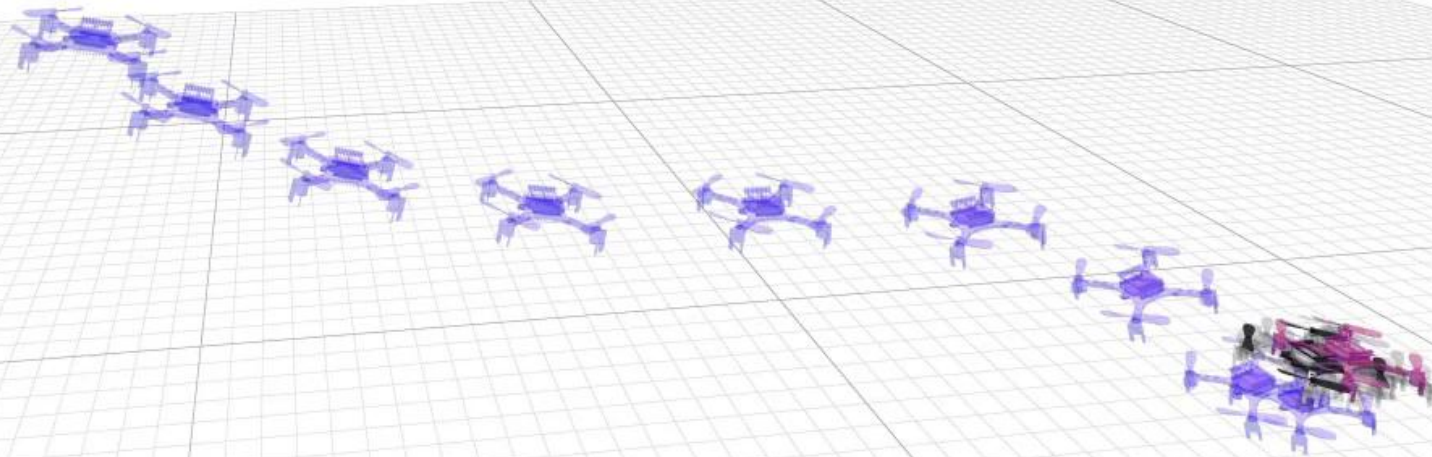


Robust MPC through Massive Parallel SQP

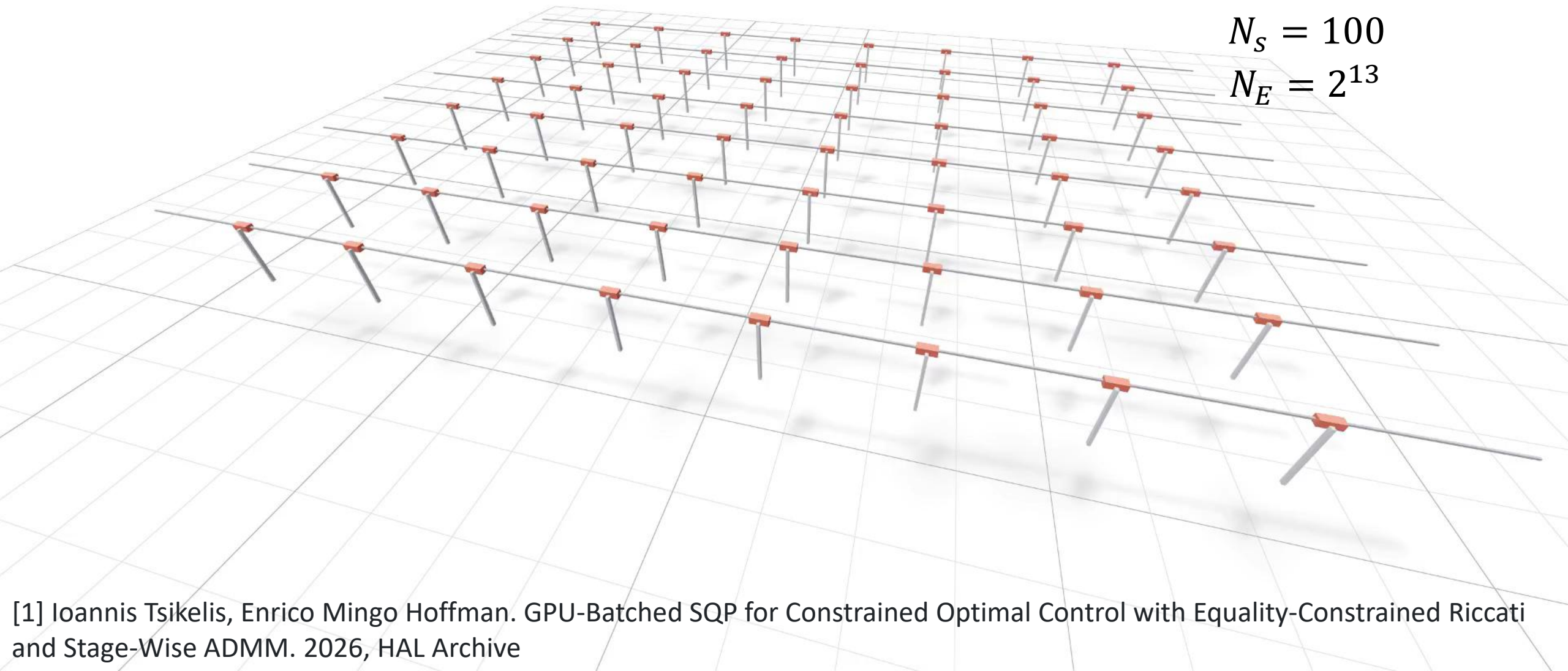


$$N_s = 100$$

$$N_E = 512$$



Robust MPC through Massive Parallel SQP



Questions???



Exercise Session

 https://github.com/hucebot/introduction_to_oc

